Assessment and Answer Guide

DEVOPS ENGINEERING MASTERY — Volume 05, Companion Volume. First Edition, 8 November 2026. This guide contains 120 MCQs, 30 short-answer questions, 20 scenario-based troubleshooting questions, 10 incident case studies, the practical final examination and the capstone rubric — followed by a separate answer key with explanations. Attempt every question yourself before reading its answer; the explanation matters more than the letter.

> **WARNING:** Do not copy assignment solutions word for word. Read the outline, close this guide, and do the work yourself in your test repository. That is where the learning happens.

মূল্যায়ন ও উত্তর নির্দেশিকা

মূল্যায়ন, উত্তর নির্দেশিকা ও কুইক রেফারেন্স — CI/CD Pipeline Engineering

এই অংশটি **DevOps Engineering Mastery — Volume 05: CI/CD Pipeline Engineering** বইয়ের মূল্যায়ন ও সহায়ক অংশ। এখানে আছে ১২০টি বহুনির্বাচনি প্রশ্ন (MCQ), ৩০টি সংক্ষিপ্ত প্রশ্ন, ২০টি পরিস্থিতিভিত্তিক সমস্যা-সমাধান প্রশ্ন, ১০টি ইনসিডেন্ট কেস স্টাডি, একটি ব্যবহারিক চূড়ান্ত পরীক্ষা, সম্পূর্ণ উত্তর নির্দেশিকা, ক্যাপস্টোন গ্রেডিং রুব্রিক, ১৫০টির বেশি পদের শব্দকোষ (Glossary), YAML রেফারেন্স, চেকলিস্ট, ট্রাবলশুটিং ডিসিশন ট্রি, দক্ষতা-ট্র্যাকিং শিট এবং অফিসিয়াল রেফারেন্স।

> **সতর্কতা — আগে চেষ্টা, তারপর উত্তর:** উত্তর নির্দেশিকা পড়ার আগে প্রতিটি প্রশ্ন নিজে চেষ্টা করুন। MCQ-এর অক্ষর মুখস্থ করলে কিছু শিখবেন না — ব্যাখ্যাটি বুঝুন, তারপর নিজের পাইপলাইনে প্রয়োগ করুন। পরীক্ষার অংশটি সময় ধরে, নোট বন্ধ রেখে দেওয়ার চেষ্টা করুন।

> **কীভাবে ব্যবহার করবেন:** প্রতিটি অধ্যায় পড়ার পর সংশ্লিষ্ট MCQ ব্লকটি সমাধান করুন। ভুল হওয়া প্রশ্নের অধ্যায়টি আবার পড়ুন এবং ল্যাব ওয়ার্কবুকের সংশ্লিষ্ট ল্যাবটি করুন। ক্যাপস্টোন শুরু করার আগে রুব্রিকটি ভালো করে পড়ুন — সেখানেই বলা আছে কোন মানদণ্ডে কত নম্বর।

মূল্যায়ন কাঠামো এক নজরে

অংশ	পরিমাণ	উদ্দেশ্য
MCQ (অংশ A)	১২০টি	২৪টি অধ্যায়ের ধারণাগত বোঝাপড়া যাচাই
সংক্ষিপ্ত প্রশ্ন (অংশ B)	৩০টি	সংক্ষেপে ব্যাখ্যা করার ক্ষমতা
পরিস্থিতিভিত্তিক প্রশ্ন (অংশ C)	২০টি	বাস্তব সমস্যায় নির্ণয় ও সিদ্ধান্ত
ইনসিডেন্ট কেস স্টাডি (অংশ D)	১০টি	বাস্তব ঘটনা থেকে শিক্ষা

চূড়ান্ত পরীক্ষা (অংশ E)	৩টি সেকশন, ১০০ নম্বর	তত্ত্ব + ডিজাইন + হাতে-কলমে কাজ
উত্তর নির্দেশিকা (অংশ F)	সব প্রশ্নের উত্তর ও ব্যাখ্যা	স্ব-মূল্যায়ন ও ক্যাপস্টোন রুব্রিক
শব্দকোষ (অংশ G)	১৬০টি পদ	বাংলা ব্যাখ্যাসহ প্রযুক্তিগত শব্দভাণ্ডার
কুইক রেফারেন্স (অংশ H–K)	রেফারেন্স, চেকলিস্ট, ডিসিশন ট্রি, ট্র্যাকিং শিট	কাজের পাশে রাখার সহায়ক

নিয়ম: এই বইয়ের সব উদাহরণে কাল্পনিক ডোমেইন (`technova.example`), TEST-NET আইপি (`203.0.113.x`) এবং কাল্পনিক প্লেসহোল্ডার (`CHANGE-ME-...`) ব্যবহার করা হয়েছে। কোনো আসল secret, token, পাসওয়ার্ড বা মূল্য এই বইয়ে নেই। সব নমুনা টার্মিনাল আউটপুট চিত্রণমূলক।

অংশ A — বহুনির্বাচনি প্রশ্ন

১২০টি বহুনির্বাচনি প্রশ্ন (MCQ)

প্রতিটি প্রশ্নের চারটি বিকল্প আছে — ক, খ, গ, ঘ। সঠিক উত্তরটি বেছে নিন। উত্তর ও ব্যাখ্যা আছে অংশ F-এ, তবে আগে নিজে চেষ্টা করুন। প্রশ্নগুলো শুধু YAML মুখস্থ পরীক্ষা করে না — কেন একটি সিদ্ধান্ত নেওয়া হয়, কোনো ব্যর্থতার কারণ কী, এবং কোন পদ্ধতি কখন উপযুক্ত, তা যাচাই করে।

ব্লক ১ — প্রশ্ন ১–২০: সফটওয়্যার ডেলিভারি, CI/CD ধারণা ও পাইপলাইন আর্কিটেকচার (অধ্যায় ১–৪)

1. সফটওয়্যার ডেলিভারি লাইফসাইকেলে Release Engineering-এর মূল কাজ কী?
   ক) ব্যবহারকারীর অভিযোগের উত্তর দেওয়া খ) শুধু কোড লেখা গ) কোডকে নির্ভরযোগ্যভাবে build, test, package ও release করার প্রক্রিয়া নকশা ও পরিচালনা করা ঘ) শুধু সার্ভার কেনা
2. Continuous Integration (CI)-এর সবচেয়ে গুরুত্বপূর্ণ উদ্দেশ্য কোনটি?
   ক) ডেটাবেস মুছে ফেলা খ) কোড দ্রুত লেখা গ) ডেভেলপারদের পরিবর্তন ঘন ঘন একত্র করে স্বয়ংক্রিয়ভাবে build ও test করে সংঘাত (conflict) ও ত্রুটি তাড়াতাড়ি ধরা ঘ) সার্ভারের খরচ কমানো
3. Continuous Delivery আর Continuous Deployment-এর মূল পার্থক্য কী?
   ক) Delivery শুধু মোবাইল অ্যাপের জন্য খ) Delivery-তে production-এ যাওয়ার আগে মানুষের অনুমোদন লাগে, Deployment-এ প্রতিটি সফল পরিবর্তন স্বয়ংক্রিয়ভাবে production-এ যায় গ) কোনো পার্থক্য নেই ঘ) Deployment-এ test লাগে না
4. একটি দল দিনে একবার বড় merge করে, তাতে প্রায়ই conflict হয়। CI দৃষ্টিকোণ থেকে সেরা পরামর্শ কী?
   ক) Git ব্যবহার বন্ধ করা খ) ছোট ছোট পরিবর্তন ঘন ঘন merge করা ও প্রতিটি merge-এ স্বয়ংক্রিয় build/test চালানো গ) শুধু শুক্রবার merge করা ঘ) আরও বড় merge করা
5. Build output, release artifact আর deployed application-এর মধ্যে পার্থক্য কী?
   ক) তিনটি একই জিনিস খ) Build output হলো কম্পাইল করা ফল, artifact হলো সংস্করণসহ সংরক্ষিত প্রকাশযোগ্য প্যাকেজ, deployed application হলো কোনো environment-এ চলমান সংস্করণ গ) Deployed application হলো সোর্স কোডের কপি ঘ) Artifact হলো সোর্স কোড
6. Pipeline-এ একটি stage ব্যর্থ হলে পরের stage চলা উচিত নয় — এই নীতিকে কী বলে?
   ক) Retry-forever খ) Fail-slow গ) Fail-fast ঘ) Ignore-error
7. Idempotency বলতে পাইপলাইনে কী বোঝায়?
   ক) পাইপলাইন একবারই চলবে খ) একই ইনপুটে একই পাইপলাইন বারবার চালালে একই ফল পাওয়া যায়, দ্বিতীয়বার চালালে ক্ষতি হয় না গ) পাইপলাইন খুব দ্রুত চলে ঘ) পাইপলাইন কখনো ব্যর্থ হয় না
8. Git-এ trunk-based development-এর সুবিধা কী?
   ক) code review লাগে না খ) কোনো test লাগে না গ) ছোট পরিবর্তন দ্রুত main branch-এ মেশে, merge conflict ও drift কমে ঘ) দীর্ঘদিন আলাদা branch রাখা যায়
9. Pull request-এ CI চালানোর মূল উদ্দেশ্য কী?
   ক) কোড সুন্দর দেখানো খ) সার্ভার বন্ধ করা গ) ডেটাবেস ব্যাকআপ নেওয়া ঘ) merge করার আগেই test, lint ও build যাচাই করে ত্রুটিপূর্ণ কোড main branch-এ ঢোকা আটকানো

10. Pipeline trigger হিসেবে `push` আর `pull_request` ইভেন্টের পার্থক্য কী?
    ক) push শাখায় সরাসরি কমিটে চলে, pull_request merge-এর আগে প্রস্তাবিত পরিবর্তনে চলে খ) কোনো পার্থক্য নেই গ) push শুধু GitLab-এ আছে ঘ) pull_request শুধু Jenkins-এ আছে
11. একটি pipeline-এ build, test, deploy — এই ক্রমটি কেন গুরুত্বপূর্ণ?
    ক) এটি শুধু প্রথা খ) deploy আগে করলে build দ্রুত হয় গ) test না পেরিয়ে deploy করলে ত্রুটিপূর্ণ কোড production-এ যাবে, তাই ক্রমটি একটি quality gate ঘ) ক্রমের কোনো গুরুত্ব নেই
12. Monorepo-তে শুধু বদলানো অংশের জন্য pipeline চালানোকে কী বলে?
    ক) Random build খ) Manual deploy গ) Path-based / selective trigger ঘ) Full build
13. Release Engineering-এ versioning কেন দরকার?
    ক) সার্ভার দ্রুত করার জন্য খ) কোন build কোথায় চলছে, কোন পরিবর্তন আছে এবং প্রয়োজনে কোন সংস্করণে ফিরতে হবে, তা নিশ্চিতভাবে জানার জন্য গ) শুধু নাম সুন্দর করার জন্য ঘ) test এড়ানোর জন্য
14. Feature branch দীর্ঘদিন main থেকে আলাদা থাকলে কী ঝুঁকি বাড়ে?
    ক) কোড দ্রুত হয় খ) merge conflict, অজানা integration ত্রুটি ও drift বাড়ে গ) সার্ভারের খরচ কমে ঘ) কোনো ঝুঁকি নেই
15. Pipeline observability বলতে কী বোঝায়?
    ক) পাইপলাইন মুছে ফেলা খ) পাইপলাইন গোপন রাখা গ) প্রতিটি run-এর log, status, সময় ও মেট্রিক দেখে কোথায়, কেন ব্যর্থ হলো তা বোঝা ঘ) পাইপলাইন দেখতে সুন্দর হওয়া
16. Git tag `v1.2.0` দিয়ে release চিহ্নিত করার সুবিধা কী?
    ক) সার্ভার বন্ধ হয় খ) test স্বয়ংক্রিয়ভাবে মুছে যায় গ) কোড বদলে যায় ঘ) নির্দিষ্ট কমিটকে স্থায়ীভাবে চিহ্নিত করা যায়, ফলে একই release বারবার তৈরি ও যাচাই করা যায়
17. CI-তে “broken build” দীর্ঘক্ষণ ভাঙা অবস্থায় রাখলে কী হয়?
    ক) build দ্রুত হয় খ) দলের সবাই অনিশ্চিত কোডের উপর কাজ করে, ত্রুটি জমে ও release আটকে যায় গ) কিছুই হয় না ঘ) খরচ কমে
18. Pipeline-এর stage-গুলোর মধ্যে dependency থাকার অর্থ কী?
    ক) একটি stage-এর ফলাফল বা artifact পরের stage-এর ইনপুট, তাই ক্রম মেনে চলতে হয় খ) সব stage একসাথে চলে গ) stage-গুলো স্বাধীন ঘ) dependency মানে কোডের লাইব্রেরি মাত্র
19. Manual deployment-এর তুলনায় automated pipeline-এর সবচেয়ে বড় সুবিধা কোনটি?
    ক) সার্ভার কেনা লাগে না খ) পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য ও ভুল কম হয়; প্রতিটি release একই ধাপে একইভাবে হয় গ) কোড লেখা লাগে না ঘ) মানুষের দরকার একদম থাকে না
20. একটি ছোট দলের জন্য প্রথম CI পাইপলাইনে কোনটি সবচেয়ে আগে রাখা উচিত?
    ক) শুধু Docker image build খ) প্রতিটি push/PR-এ install, lint ও unit test — সহজ, দ্রুত ও নির্ভরযোগ্য গেট গ) শুধু production deploy ঘ) জটিল canary deployment

ব্লক ২ — প্রশ্ন ২১–৪০: Build, Artifact ও GitHub Actions ভিত্তি (অধ্যায় ৫–৮)

1. Artifact repository (যেমন npm registry, container registry) কেন দরকার?
    ক) লগ সংরক্ষণের জন্য খ) সোর্স কোড সংরক্ষণের জন্য গ) build করা সংস্করণসহ প্যাকেজ সংরক্ষণ, খুঁজে পাওয়া ও একই artifact বারবার deploy করার জন্য ঘ) পাসওয়ার্ড সংরক্ষণের জন্য

2. `package-lock.json` কমিট করার মূল কারণ কী?
   ক) npm দ্রুত করা খ) ফাইল বড় দেখানো গ) সব পরিবেশে একই dependency সংস্করণ ইনস্টল নিশ্চিত করা, build পুনরাবৃত্তিযোগ্য করা ঘ) কোড গোপন রাখা
3. GitHub Actions-এ workflow, job ও step-এর সম্পর্ক কী?
   ক) Step-এ একাধিক workflow থাকে খ) তিনটি একই গ) Workflow-এ একাধিক job থাকে, প্রতিটি job-এ একাধিক step থাকে; job ডিফল্টভাবে সমান্তরালে চলে ঘ) Job শুধু একটিই হতে পারে
4. GitHub-hosted runner আর self-hosted runner-এর মূল পার্থক্য কী?
   ক) Hosted runner-এ Docker চলে না খ) কোনো পার্থক্য নেই গ) Self-hosted runner বিনামূল্যে সীমাহীন ঘ) Hosted runner GitHub পরিচালনা করে ও প্রতিবার নতুন পরিবেশ দেয়; self-hosted runner নিজের মেশিনে চলে, নিয়ন্ত্রণ বেশি কিন্তু রক্ষণাবেক্ষণ ও নিরাপত্তার দায়িত্ব নিজের
5. Self-hosted runner-এ public repository-র pull request চালানো কেন ঝুঁকিপূর্ণ?
   ক) ঝুঁকি নেই খ) GitHub তা নিষিদ্ধ করেছে গ) runner ধীর হয় ঘ) অপরিচিত কোড আপনার নেটওয়ার্ক ও secret-এর কাছাকাছি পরিবেশে চলতে পারে, তাই isolation দরকার
6. GitHub Actions-এ `actions/checkout@v4`-এর কাজ কী?
   ক) কোড মুছে ফেলা খ) secret তৈরি করা গ) repository-র কোড runner-এ নামিয়ে আনা, যাতে build/test করা যায় ঘ) সার্ভার deploy করা
7. Reusable workflow আর composite action-এর পার্থক্য কী?
   ক) Reusable workflow পুরো workflow অন্য workflow থেকে ডাকে (job স্তরে), composite action একাধিক step একত্র করে একটি step হিসেবে ব্যবহার করে খ) Reusable workflow শুধু একবার ব্যবহার করা যায় গ) Composite action শুধু Jenkins-এ চলে ঘ) একই জিনিস
8. `needs:` ব্যবহারের উদ্দেশ্য কী?
   ক) runner বদলানো খ) secret লুকানো গ) একটি job অন্য job-এর উপর নির্ভরশীল করা — আগের job সফল না হলে পরেরটি চলবে না, এবং তার output ব্যবহার করা ঘ) job দ্রুত করা
9. Matrix strategy কেন ব্যবহার করা হয়?
   ক) একটি job একাধিক Node সংস্করণ বা OS-এ সমান্তরালে চালিয়ে সামঞ্জস্য (compatibility) যাচাই করতে খ) খরচ কমাতে গ) deploy এড়াতে ঘ) log লুকাতে
10. GitHub Actions-এ `if: failure()` কী করে?
    ক) আগের step ব্যর্থ হলেই চলে — যেমন ব্যর্থতার নোটিফিকেশন পাঠানো খ) শুধু সফল হলে চলে গ) সবসময় চলে ঘ) কখনো চলে না
11. `GITHUB_TOKEN`-এর অনুমতি (permissions) সীমিত রাখা কেন জরুরি?
    ক) token কাজ করে না খ) token দ্রুত শেষ হয় গ) GitHub বাধ্য করে ঘ) Least privilege — আপস হলে ক্ষতি সীমিত থাকে; শুধু প্রয়োজনীয় অনুমতিই দেওয়া উচিত
12. Workflow-এ third-party action ব্যবহারের আগে কী যাচাই করা উচিত?
    ক) শুধু তারকা সংখ্যা খ) শুধু নাম সুন্দর কি না গ) কিছুই না ঘ) action-টির উৎস, সংস্করণ pin করা হয়েছে কি না, এবং এটি কী অনুমতি চায়
13. Action-কে commit SHA দিয়ে pin করার সুবিধা কী?
    ক) secret নিরাপদ হয় খ) tag বদলে গেলেও একই কোড চলবে — supply-chain আক্রমণে হঠাৎ বদলে যাওয়া কোড ঢোকার ঝুঁকি কমে গ) action দ্রুত হয় ঘ) কোড ছোট হয়

14. GitHub Actions-এ caching (`actions/cache`) কখন সবচেয়ে কার্যকর?
    ক) secret সংরক্ষণে খ) log সংরক্ষণে গ) `node_modules`-এর মতো বড়, বারবার একই থাকা dependency সংরক্ষণ করে install সময় কমাতে ঘ) প্রতিবার সবকিছু মুছে ফেললে
15. Cache key-তে lockfile-এর hash ব্যবহার করা হয় কেন?
    ক) cache মুছে ফেলতে খ) dependency বদলালে পুরনো cache ব্যবহার না হয়ে নতুন cache তৈরি হয় গ) runner বদলাতে ঘ) দেখতে ভালো লাগে
16. Artifact upload (`actions/upload-artifact`) কখন ব্যবহার করবেন?
    ক) কোড মুছতে খ) একটি job-এর build করা ফাইল অন্য job-এ বা পরে ডাউনলোডের জন্য সংরক্ষণ করতে গ) log লুকাতে ঘ) secret পাঠাতে
17. Workflow-এ `concurrency` ব্যবহারের উদ্দেশ্য কী?
    ক) run দ্রুত করা খ) log বাড়ানো গ) একই branch-এ একাধিক run একসাথে চলা আটকে deploy conflict ও race condition কমানো ঘ) runner সংখ্যা বাড়ানো
18. `pull_request_target` ইভেন্ট কেন সতর্কতার সঙ্গে ব্যবহার করতে হয়?
    ক) এটি base repository-র secret ও অনুমতি নিয়ে চলে, তাই অপরিচিত PR কোডে checkout করলে secret ফাঁসের ঝুঁকি থাকে খ) এটি কাজ করে না গ) এটি ধীর ঘ) এটি শুধু private repo-তে চলে
19. GitHub Actions-এ environment protection rule-এর কাজ কী?
    ক) runner মুছে ফেলা খ) production environment-এ deploy-এর আগে নির্দিষ্ট reviewer-এর অনুমোদন বাধ্যতামূলক করা গ) test বন্ধ করা ঘ) কোড সুন্দর করা
20. Workflow run ব্যর্থ হলে প্রথমে কী দেখা উচিত?
    ক) কোন job/step ব্যর্থ হয়েছে, তার log-এর ত্রুটি বার্তা ও কোন কমান্ডে থেমেছে খ) সার্ভার রিস্টার্ট করা গ) secret বদলে ফেলা ঘ) পুরো workflow মুছে ফেলা

ব্লক ৩ — প্রশ্ন ৪১–৬০: Secrets, OIDC, Optimization, GitLab CI/CD (অধ্যায় ৯–১৩)

1. GitHub Actions-এ secret কখনো log-এ print করা উচিত নয় কেন?
    ক) secret কাজ করা বন্ধ করে খ) log বড় হয় গ) log সংরক্ষিত থাকে ও অনেকের কাছে দৃশ্যমান হতে পারে, ফলে secret ফাঁস হয় ঘ) GitHub নিষেধ করে না
2. OIDC (OpenID Connect) দিয়ে ক্লাউডে authenticate করার সুবিধা কী?
    ক) secret log-এ দেখা যায় খ) কোনো অনুমতি লাগে না গ) token কখনো মেয়াদোত্তীর্ণ হয় না ঘ) দীর্ঘমেয়াদি secret সংরক্ষণ করতে হয় না; প্রতিটি run-এর জন্য স্বল্পমেয়াদি token তৈরি হয়
3. Workflow-এ `on.push.branches` সীমিত করার সুবিধা কী?
    ক) runner বাড়ে খ) run কমে খরচ ও সময় বাঁচে, অপ্রয়োজনীয় branch-এ pipeline চলে না গ) কোড দ্রুত হয় ঘ) test বন্ধ হয়
4. Expression `${{ github.event.pull_request.number }}` কী বোঝায়?
    ক) একটি ফাইলের পথ খ) বর্তমান pull request-এর নম্বর — context থেকে পাওয়া তথ্য গ) একটি secret ঘ) runner-এর নাম
5. GitLab CI/CD-তে `.gitlab-ci.yml` ফাইলের কাজ কী?
    ক) pipeline-এর stage, job, script ও নিয়ম ঘোষণা করা খ) ডেটাবেস তৈরি করা গ) কোড লেখা ঘ) সার্ভার ইনস্টল করা

6. GitLab-এ stage আর job-এর সম্পর্ক কী?
   ক) Stage হলো ধাপের ক্রম (যেমন build → test → deploy), job হলো সেই stage-এর ভেতরে চলা কাজ; একই stage-এর job সমান্তরালে চলে খ) Stage শুধু একটিই হয় গ) Job-এ stage থাকে ঘ) একই
7. GitLab Runner-এর executor (Shell বনাম Docker) বেছে নেওয়ার ক্ষেত্রে মূল বিবেচনা কী?
   ক) নাম সুন্দর হওয়া খ) Docker executor-এ cache চলে না গ) Docker executor প্রতিটি job-এ পরিষ্কার পরিবেশ দেয় ও isolation ভালো; Shell executor সহজ কিন্তু পরিবেশ দূষণের ঝুঁকি থাকে ঘ) Shell executor সবসময় দ্রুত
8. GitLab-এ `rules:` ব্যবহার করা হয় কেন?
   ক) runner ইনস্টল করতে খ) কোন branch, tag বা পরিবর্তনে job চলবে বা চলবে না, তা নিয়ন্ত্রণ করতে গ) secret তৈরি করতে ঘ) কোড সুন্দর করতে
9. GitLab-এ `needs:` আর `dependencies:`-এর পার্থক্য কী?
   ক) needs শুধু deploy-এ চলে খ) dependencies শুধু test-এ চলে গ) needs দিয়ে stage-এর ক্রম ভেঙে আগের job শেষ হলেই পরের job চালানো যায়; dependencies দিয়ে কোন job-এর artifact নেওয়া হবে তা নিয়ন্ত্রণ করা হয় ঘ) একই
10. GitLab-এ protected variable বলতে কী বোঝায়?
    ক) শুধু নামের variable খ) মুছে ফেলা যায় না এমন variable গ) শুধু protected branch/tag-এর pipeline-এ ব্যবহারযোগ্য variable — অপরিচিত branch-এ secret ফাঁস রোধ করে ঘ) সবাই দেখতে পারে এমন variable
11. GitLab environment-এর “stop” action কেন থাকে?
    ক) deploy বন্ধ করতে খ) review app বা অস্থায়ী environment মুছে সম্পদ (resource) মুক্ত করতে গ) pipeline মুছতে ঘ) কোড মুছতে
12. GitLab-এ `include:` ব্যবহারের সুবিধা কী?
    ক) runner বাড়ানো খ) secret লুকানো গ) ফাইল বড় করা ঘ) একাধিক project-এ একই pipeline অংশ পুনঃব্যবহার করা, রক্ষণাবেক্ষণ সহজ করা
13. GitLab pipeline-এ cache আর artifact-এর পার্থক্য কী?
    ক) একই খ) Cache হলো পরের run দ্রুত করার জন্য অস্থায়ী সংরক্ষণ, artifact হলো job-এর ফল যা অন্য job বা পরে ডাউনলোডের জন্য সংরক্ষিত হয় গ) Cache কখনো মুছে না ঘ) Artifact শুধু log-এ থাকে
14. GitLab-এ manual job (`when: manual`) কখন ব্যবহার করা হয়?
    ক) build দ্রুত করতে খ) production deploy-এর মতো সংবেদনশীল ধাপে মানুষের অনুমোদন নিতে গ) সব job-এ ঘ) test এড়াতে
15. GitLab Runner shared বনাম specific — কখন specific runner দরকার?
    ক) specific runner বিনামূল্যে খ) সবসময় shared ভালো গ) বিশেষ পরিবেশ, GPU, নির্দিষ্ট নেটওয়ার্ক বা নিরাপত্তা প্রয়োজন হলে নির্দিষ্ট project-এর জন্য আলাদা runner ঘ) কখনো না
16. Workflow optimization-এ সবচেয়ে বড় লাভ সাধারণত কোথা থেকে আসে?
    ক) log মুছে ফেললে খ) secret বাড়ালে গ) runner-এর নাম বদলালে ঘ) অপ্রয়োজনীয় job বাদ দেওয়া, cache করা ও সমান্তরাল (parallel) job চালানো থেকে
17. Pipeline-এ flaky test বলতে কী বোঝায়?
    ক) সবসময় পাস করা test খ) কোড না বদলালেও কখনো পাস, কখনো ফেল করা test — সাধারণত timing বা বাইরের নির্ভরতার কারণে গ) ধীর test ঘ) লেখা হয়নি এমন test

18. GitLab-এ `workflow:rules` দিয়ে কী নিয়ন্ত্রণ করা হয়?
    ক) runner-এর গতি খ) secret-এর মেয়াদ গ) কোডের স্টাইল ঘ) পুরো pipeline কখন তৈরি হবে (যেমন শুধু merge request বা শুধু main branch-এ)
19. GitHub Actions-এ `actions/setup-node`-এর cache অপশন কী করে?
    ক) npm dependency cache করে `npm ci` দ্রুত করে খ) Docker image তৈরি করে গ) secret সংরক্ষণ করে ঘ) কোড মুছে ফেলে
20. একটি pipeline ২০ মিনিট ধরে চলে, বেশিরভাগ সময় dependency install-এ যায়। প্রথম পদক্ষেপ কী?
    ক) lockfile-ভিত্তিক cache চালু করা ও `npm ci` ব্যবহার করা খ) test মুছে ফেলা গ) workflow মুছে ফেলা ঘ) runner বড় করা

ব্লক ৪ — প্রশ্ন ৬১–৮০: Jenkins, Testing ও Quality Gates (অধ্যায় ১৪–১৯)

1. Jenkins-এ controller আর agent-এর ভূমিকা কী?
   ক) Controller build চালায়, agent কিছু করে না খ) Agent controller-কে নিয়ন্ত্রণ করে গ) দুটোই একই কাজ করে ঘ) Controller pipeline পরিচালনা, সময়সূচি ও UI দেয়; agent আসল build/test কাজ চালায়
2. Jenkins controller-এ সরাসরি build চালানো কেন নিরুৎসাহিত করা হয়?
   ক) build controller-এর সম্পদ খেয়ে ফেলে, নিরাপত্তা ঝুঁকি বাড়ে ও controller অস্থিতিশীল হয় খ) Jenkins তা নিষিদ্ধ করেছে গ) build ধীর হয় ঘ) agent কাজ করে না
3. Declarative Pipeline আর Scripted Pipeline-এর মূল পার্থক্য কী?
   ক) কোনো পার্থক্য নেই খ) Declarative-এ Groovy লেখা যায় না একদম গ) Scripted শুধু GitLab-এ চলে ঘ) Declarative-তে নির্দিষ্ট গঠন (stages, steps, post) থাকে ও পড়া সহজ; Scripted-তে Groovy-র পূর্ণ নমনীয়তা থাকে কিন্তু জটিলতা বেশি
4. Jenkinsfile-এ `post { failure { ... } }` ব্লকের কাজ কী?
   ক) build শুরু করা খ) secret তৈরি করা গ) pipeline ব্যর্থ হলে নোটিফিকেশন বা cleanup চালানো ঘ) সবসময় চলে
5. Jenkins-এ credentials কীভাবে সংরক্ষণ করা উচিত?
   ক) Git-এ কমিট করা খ) Jenkinsfile-এ সরাসরি লেখা গ) চ্যাটে শেয়ার করা ঘ) Jenkins Credentials Store-এ সংরক্ষণ করে ID দিয়ে ব্যবহার করা, কখনো কোডে বা log-এ প্রকাশ না করা
6. Jenkins Shared Library-এর সুবিধা কী?
   ক) agent বাড়ানো খ) কোড বড় করা গ) একাধিক pipeline-এ একই ধাপ পুনঃব্যবহার করা, পরিবর্তন এক জায়গায় করা ঘ) secret সংরক্ষণ করা
7. Jenkins plugin ইনস্টল করার আগে কী যাচাই করা উচিত?
   ক) শুধু নাম খ) plugin-টি রক্ষণাবেক্ষণ করা হয় কি না, নিরাপত্তা সতর্কতা আছে কি না, এবং Jenkins সংস্করণের সঙ্গে সামঞ্জস্য গ) কিছুই না ঘ) শুধু ডাউনলোড সংখ্যা
8. Jenkins pipeline-এ `agent { docker { image 'node:20' } }` বলতে কী বোঝায়?
   ক) Jenkins মুছে ফেলা খ) ওই stage-টি নির্দিষ্ট Docker image-এর ভেতরে চালানো — পরিবেশ সামঞ্জস্যপূর্ণ থাকে গ) Docker ইনস্টল করা ঘ) সার্ভার deploy করা
9. Unit test আর integration test-এর পার্থক্য কী?
   ক) Unit test একটি ছোট অংশ আলাদাভাবে যাচাই করে; integration test একাধিক অংশ একসাথে কাজ করে কি না যাচাই করে খ) Integration test দ্রুত হয় গ) একই ঘ) Unit test-এ ডেটাবেস লাগে

10. End-to-end (E2E) test-এর সীমাবদ্ধতা কী?
    ক) এটি কখনো ব্যর্থ হয় না খ) কোনো সীমাবদ্ধতা নেই গ) এটি unit test-এর বিকল্প ঘ) ধীর, ভঙ্গুর (flaky) হতে পারে ও রক্ষণাবেক্ষণ ব্যয়বহুল — তাই অল্প সংখ্যায় গুরুত্বপূর্ণ প্রবাহে ব্যবহার করা উচিত
11. Code coverage ৯০% হলেই কি কোড নির্ভুল ধরা যায়?
    ক) হ্যাঁ, test লেখা বন্ধ করা যায় খ) না, coverage অর্থহীন গ) না — coverage শুধু কোন লাইন চলেছে তা বলে, যুক্তি সঠিক কি না তা বলে না ঘ) হ্যাঁ, সম্পূর্ণ নিরাপদ
12. ESLint-এর মতো linting pipeline-এ রাখার উদ্দেশ্য কী?
    ক) স্টাইল, সম্ভাব্য ভুল ও অসামঞ্জস্য তাড়াতাড়ি ধরা, দলের কোড একরূপ রাখা খ) test বদলে দেওয়া গ) deploy করা ঘ) কোড চালানো
13. Dependency vulnerability scanning (যেমন `npm audit`) কী ধরে?
    ক) কোডের স্টাইল ভুল খ) ব্যবহৃত লাইব্রেরিতে জানা নিরাপত্তা দুর্বলতা আছে কি না গ) সার্ভারের গতি ঘ) টাইপ ভুল
14. Secret scanning কেন pipeline-এ দরকার?
    ক) test দ্রুত করতে খ) কোডে ভুল করে কমিট করা token বা পাসওয়ার্ড merge-এর আগে ধরা গ) secret তৈরি করতে ঘ) log সুন্দর করতে
15. Quality gate বলতে কী বোঝায়?
    ক) নির্দিষ্ট মান (test পাস, coverage, scan পরিষ্কার) না পেরোলে পরের ধাপে যাওয়া যাবে না — এমন শর্ত খ) একটি দরজা গ) deploy-এর পরের ধাপ ঘ) কোড review-এর বিকল্প
16. SAST আর DAST-এর পার্থক্য কী?
    ক) একই খ) SAST শুধু Docker-এ চলে গ) SAST কোড না চালিয়ে বিশ্লেষণ করে; DAST চলমান অ্যাপ্লিকেশন পরীক্ষা করে ঘ) DAST শুধু unit test-এ চলে
17. TypeScript type checking pipeline-এ রাখলে কী লাভ হয়?
    ক) test লাগে না খ) কোড দ্রুত চলে গ) build বন্ধ হয় ঘ) runtime-এর আগেই টাইপ-সংক্রান্ত ভুল ধরা পড়ে
18. Pipeline-এ test parallel চালানোর ঝুঁকি কী?
    ক) test-গুলো একই ডেটাবেস বা ফাইল ভাগ করলে একে অপরের ফল নষ্ট করতে পারে — isolation দরকার খ) test দ্রুত হয় না গ) coverage কমে যায় ঘ) কোনো ঝুঁকি নেই
19. Jenkins-এ pipeline parameter (`parameters`) কেন ব্যবহার করা হয়?
    ক) একই pipeline বিভিন্ন ইনপুট (যেমন environment বা সংস্করণ) দিয়ে চালানো খ) secret সংরক্ষণ করতে গ) কোড লুকাতে ঘ) agent মুছতে
20. Test pyramid অনুযায়ী কোন test সবচেয়ে বেশি থাকা উচিত?
    ক) সব test সমান সংখ্যায় খ) Unit test — দ্রুত, সস্তা ও অনেক; তারপর integration, সবচেয়ে কম E2E গ) E2E test ঘ) Manual test

ব্লক ৫ — প্রশ্ন ৮১–১০০: Docker, Registry, Release ও Deployment কৌশল (অধ্যায় ২০–২২)

1. Multi-stage Docker build-এর মূল সুবিধা কী?
    ক) build ধীর হয় খ) build-এর সরঞ্জাম চূড়ান্ত image-এ না রেখে ছোট, নিরাপদ production image তৈরি করা যায় গ) Dockerfile ছোট হয় না ঘ) image বড় হয়

2. Docker image tag `latest` ব্যবহারের ঝুঁকি কী?
   ক) কোনো ঝুঁকি নেই খ) `latest` পরিবর্তনশীল (mutable) — আজকের `latest` আর গতকালের `latest` এক নয়, ফলে কোন সংস্করণ চলছে তা নিশ্চিত করা যায় না গ) registry তা গ্রহণ করে না ঘ) image মুছে যায়
3. Image digest (যেমন `sha256:...`) ব্যবহারের সুবিধা কী?
   ক) পড়া সহজ খ) tag লাগে না গ) digest অপরিবর্তনীয় — ঠিক একই image-ই সবসময় টানা হয়, পুনরাবৃত্তিযোগ্যতা নিশ্চিত হয় ঘ) image ছোট হয়
4. Docker image scan (যেমন Trivy) কী খোঁজে?
   ক) image-এর ভেতরে থাকা OS প্যাকেজ ও লাইব্রেরিতে জানা দুর্বলতা (CVE) খ) নেটওয়ার্ক সমস্যা গ) কোডের স্টাইল ঘ) সার্ভারের গতি
5. Registry-তে push করার আগে authenticate করতে হয় কেন?
   ক) শুধু অনুমোদিত ব্যক্তি বা pipeline-ই image প্রকাশ করতে পারে, অবৈধ image ঢোকা আটকায় খ) দেখতে ভালো লাগে গ) tag মুছে যায় ঘ) image বড় হয়
6. Immutable artifact বলতে কী বোঝায়?
   ক) artifact মুছে ফেলা যায় না খ) artifact বদলানো যায় না — একবার তৈরি হলে একই সংস্করণ সব environment-এ ব্যবহৃত হয় গ) artifact ছোট হয় ঘ) artifact শুধু একবার deploy করা যায়
7. Semantic Versioning (SemVer) `1.4.2`-এ প্রতিটি অংশের অর্থ কী?
   ক) কোনো অর্থ নেই খ) বছর.মাস.দিন গ) সার্ভার.ডেটাবেস.ক্যাশ ঘ) major.minor.patch — বড় পরিবর্তন, নতুন ফিচার, ছোট সংশোধন
8. Environment promotion বলতে কী বোঝায়?
   ক) সার্ভার কেনা খ) একই artifact dev → staging → production-এ ধাপে ধাপে যাচাই করে এগিয়ে নেওয়া, প্রতিটি ধাপে নতুন করে build না করা গ) ডেটাবেস মুছে ফেলা ঘ) কোড আবার লেখা
9. Blue-green deployment-এর মূল ধারণা কী?
   ক) সার্ভার বন্ধ করে deploy করা খ) দুটি সার্ভার একসাথে চালিয়ে নতুন সংস্করণ প্রস্তুত করে ট্রাফিক একবারে বদলে দেওয়া; সমস্যা হলে পুরনো সংস্করণে ফেরা সহজ গ) অর্ধেক ব্যবহারকারীকে নতুন সংস্করণ দেওয়া ঘ) কোড দুই ভাগে ভাগ করা
10. Canary deployment কখন উপযুক্ত?
    ক) ডেটাবেস না থাকলে খ) test না থাকলে গ) নতুন সংস্করণ প্রথমে অল্প ব্যবহারকারীর উপর পরীক্ষা করে মেট্রিক দেখে ধীরে ধীরে বাড়াতে চাইলে ঘ) সবসময়
11. Rolling deployment-এর ঝুঁকি কী?
    ক) rollback অসম্ভব খ) কোনো ঝুঁকি নেই গ) কিছুক্ষণ পুরনো ও নতুন সংস্করণ একসাথে চলে — সংস্করণের মধ্যে অসামঞ্জস্য থাকলে সমস্যা হতে পারে ঘ) সার্ভার বন্ধ থাকে
12. Feature flag-এর সুবিধা কী?
    ক) কোড মুছে ফেলা খ) সার্ভার দ্রুত করা গ) deploy আর release আলাদা করা যায় — কোড production-এ গেলেও ফিচারটি পরে চালু বা বন্ধ করা যায় ঘ) test বন্ধ করা
13. Database migration deploy-এর সময় সবচেয়ে নিরাপদ নীতি কোনটি?
    ক) ডেটাবেস মুছে নতুন করে তৈরি করা খ) Expand-and-contract — প্রথমে নতুন কলাম/ক্ষেত্র যোগ করা, কোড সামঞ্জস্য করা, পরে পুরনোটি সরানো গ) কলাম একবারে মুছে ফেলা ঘ) migration ছাড়া deploy করা
14. Health check endpoint (`/health`) deploy-এ কেন দরকার?
    ক) নতুন সংস্করণ সত্যিই অনুরোধ নিতে প্রস্তুত কি না যাচাই করে তবেই ট্রাফিক দেওয়া যায় খ) দেখতে ভালো লাগে গ) log বাড়ানো ঘ) secret যাচাই করা

15. Rollback পরিকল্পনা deploy-এর আগেই থাকা উচিত কেন?
    ক) সমস্যা হলে দ্রুত ও চাপ ছাড়া আগের ভালো সংস্করণে ফেরা যায়; ঘটনার সময় পরিকল্পনা বানানো ধীর ও ভুলপ্রবণ খ) deploy দ্রুত হয় গ) rollback কখনো লাগে না ঘ) test কম লাগে
16. Release candidate (RC) বলতে কী বোঝায়?
    ক) চূড়ান্ত release-এর আগের পরীক্ষামূলক সংস্করণ, যা প্রায় চূড়ান্ত কিন্তু যাচাই চলছে খ) test সংস্করণ গ) পুরনো release ঘ) ব্যর্থ release
17. Approval gate staging থেকে production-এ যাওয়ার আগে কী নিশ্চিত করে?
    ক) সার্ভার বন্ধ খ) কোড সুন্দর গ) দায়িত্বশীল ব্যক্তি staging যাচাই, release note ও ঝুঁকি দেখে অনুমোদন দিয়েছেন ঘ) test মুছে ফেলা
18. Docker layer caching কখন কাজ করে?
    ক) tag বদলালে খ) Dockerfile-এর আগের layer-গুলো না বদলালে সেগুলো আবার তৈরি না করে পুনঃব্যবহার করা হয় গ) image মুছে ফেললে ঘ) সবসময়
19. `.dockerignore` ফাইলের কাজ কী?
    ক) image মুছে ফেলা খ) build context থেকে অপ্রয়োজনীয় ফাইল (যেমন `node_modules`, `.git`) বাদ দিয়ে build দ্রুত ও নিরাপদ করা গ) Dockerfile লুকানো ঘ) registry বদলানো
20. Production deploy-এর পর “smoke test” বলতে কী বোঝায়?
    ক) সার্ভার ধোঁয়া পরীক্ষা করা খ) পুরো test suite চালানো গ) মূল প্রবাহগুলো (লগইন, হোমপেজ, API) দ্রুত যাচাই করা যে deploy-এর পর অ্যাপ বেঁচে আছে ঘ) log মুছে ফেলা

ব্লক ৬ — প্রশ্ন ১০১–১২০: MERN এন্ড-টু-এন্ড, ট্রাবলশুটিং ও গভর্ন্যান্স (অধ্যায় ২৩–২৪)

1. MERN অ্যাপের CI পাইপলাইনে frontend ও backend আলাদা job-এ ভাগ করার সুবিধা কী?
    ক) দুটি অংশ সমান্তরালে test/build হয়, ব্যর্থতা আলাদাভাবে ধরা যায় খ) কোড বড় হয় গ) deploy বন্ধ হয় ঘ) খরচ বাড়ে
2. MERN অ্যাপে MongoDB-সহ integration test pipeline-এ চালাতে হলে কী দরকার?
    ক) test বাদ দেওয়া খ) production ডেটাবেস ব্যবহার করা গ) ডেটাবেস মুছে ফেলা ঘ) service container বা অস্থায়ী MongoDB দিয়ে বিচ্ছিন্ন (isolated) test পরিবেশ তৈরি করা
3. Pipeline-এ `npm ci` বনাম `npm install` — CI-তে কোনটি উপযুক্ত ও কেন?
    ক) `npm ci` — lockfile অনুযায়ী পরিষ্কারভাবে ইনস্টল করে, পুনরাবৃত্তিযোগ্য build দেয় খ) কোনোটিই নয় গ) `npm install` — দ্রুত ঘ) দুটোই একই
4. Pipeline হঠাৎ ধীর হয়ে গেলে প্রথমে কী যাচাই করা উচিত?
    ক) runner মুছে ফেলা খ) secret বদলানো গ) সব job মুছে ফেলা ঘ) কোন step সবচেয়ে বেশি সময় নিচ্ছে — install, test, নাকি build — তা log ও সময় থেকে চিহ্নিত করা
5. “Works on my machine” সমস্যা pipeline কীভাবে কমায়?
    ক) সবার কোড একই নির্দিষ্ট পরিবেশে (Docker বা নির্দিষ্ট runner image) build/test হয়, তাই পরিবেশের পার্থক্য ধরা পড়ে খ) কমায় না গ) কোড মুছে ফেলে ঘ) ডেভেলপারের মেশিন বন্ধ করে
6. Pipeline governance বলতে কী বোঝায়?
    ক) pipeline দ্রুত করা খ) log লুকানো গ) কে pipeline বদলাতে পারে, কোন অনুমোদন লাগে, secret কীভাবে ব্যবহৃত হয় — এসব নিয়ম ও নিয়ন্ত্রণ ঘ) pipeline মুছে ফেলা

7. একজন ডেভেলপার pipeline YAML বদলে secret print করার step যোগ করল। এটি আটকাতে কী সাহায্য করে?
   ক) Code review, branch protection, secret scanning ও ন্যূনতম অনুমতি (least privilege) খ) cache বন্ধ করা গ) test বাড়ানো ঘ) কিছুই না
8. Production pipeline-এ audit trail কেন দরকার?
   ক) log সুন্দর করতে খ) কে, কখন, কী deploy করল ও কোন অনুমোদন ছিল — তা পরে যাচাই ও তদন্তের জন্য সংরক্ষিত থাকে গ) সার্ভার দ্রুত করতে ঘ) খরচ কমাতে
9. Pipeline-এ retry নীতি কখন ক্ষতিকর হতে পারে?
   ক) deploy বা payment-এর মতো পার্শ্বপ্রতিক্রিয়াযুক্ত (side-effect) ধাপে অন্ধভাবে retry করলে একই কাজ দুবার হতে পারে খ) build-এ retry করলে গ) test-এ retry করলে ঘ) কখনো না
10. Disaster recovery দৃষ্টিকোণ থেকে pipeline সংজ্ঞা (YAML/Jenkinsfile) কোথায় থাকা উচিত?
    ক) কোথাও না খ) Git-এ সংস্করণসহ — হারিয়ে গেলে আবার তৈরি করা যায় গ) চ্যাটে ঘ) শুধু একজন ডেভেলপারের ল্যাপটপে
11. একটি দল GitHub Actions, GitLab ও Jenkins-এর মধ্যে বেছে নিচ্ছে। সিদ্ধান্তের সবচেয়ে গুরুত্বপূর্ণ বিবেচনা কী?
    ক) সবচেয়ে নতুন টুল খ) সবচেয়ে ব্যয়বহুল টুল গ) কোনটির লোগো সুন্দর ঘ) দলের বিদ্যমান প্ল্যাটফর্ম, রক্ষণাবেক্ষণের ক্ষমতা, নিরাপত্তা প্রয়োজন ও খরচ — কোনো একক “সেরা” টুল নেই
12. Pipeline-এর cost নিয়ন্ত্রণে সবচেয়ে কার্যকর পদক্ষেপ কোনটি?
    ক) অপ্রয়োজনীয় trigger কমানো, cache করা, ছোট runner ব্যবহার ও ব্যর্থ run তাড়াতাড়ি থামানো খ) সব test বন্ধ করা গ) secret বাড়ানো ঘ) log বাড়ানো
13. On-call প্রকৌশলী রাতে pipeline ব্যর্থতার alert পেলেন। প্রথম কাজ কী?
    ক) সার্ভার বন্ধ করা খ) সঙ্গে সঙ্গে production deploy করা গ) ব্যর্থতার পরিধি বোঝা — এটি কি একটি PR-এর test ব্যর্থতা, নাকি production deploy আটকে গেছে — তারপর runbook অনুযায়ী কাজ করা ঘ) সব secret বদলানো
14. Blameless postmortem বলতে কী বোঝায়?
    ক) ঘটনার পর দোষারোপ না করে প্রক্রিয়া ও ব্যবস্থার দুর্বলতা খুঁজে সংশোধন করা খ) কাউকে শাস্তি দেওয়া গ) ঘটনা ভুলে যাওয়া ঘ) কাউকে দোষ না দেওয়া
15. Pipeline-এ “shift left” বলতে কী বোঝায়?
    ক) নিরাপত্তা ও quality পরীক্ষা যত তাড়াতাড়ি সম্ভব (কোড লেখা ও PR পর্যায়ে) নিয়ে আসা, শেষ মুহূর্তে না রাখা খ) কোড বামে সরানো গ) deploy বামে সরানো ঘ) test পরে করা
16. একটি pipeline-এ একই commit দুবার deploy হয়ে গেল। সম্ভাব্য কারণ কী?
    ক) runner দ্রুত খ) কোডের ভুল গ) concurrency নিয়ন্ত্রণ না থাকা বা একই trigger দুবার চলা (যেমন push ও tag একসাথে) ঘ) secret ভুল
17. Release note-এ কী থাকা উচিত?
    ক) কী বদলেছে, কী ঝুঁকি আছে, কীভাবে rollback করতে হয় ও কী যাচাই করতে হয় খ) শুধু সংস্করণ নম্বর গ) শুধু ডেভেলপারের নাম ঘ) শুধু তারিখ
18. Production-এ নতুন deploy-এর পর error rate হঠাৎ বাড়ল। সেরা প্রথম পদক্ষেপ কী?
    ক) ট্রাফিক বা rollback-এর সিদ্ধান্ত নেওয়া — প্রস্তুত rollback পথে আগের স্থিতিশীল সংস্করণে ফেরা ও তদন্ত করা খ) অপেক্ষা করা ও আশা করা গ) log মুছে ফেলা ঘ) আরও deploy করা
19. Pipeline documentation-এ কী থাকা উচিত?
    ক) কিছুই না খ) আর্কিটেকচার, trigger, secret ব্যবস্থাপনা, অনুমোদন প্রক্রিয়া, rollback ধাপ ও যোগাযোগের পথ গ) শুধু পাসওয়ার্ড ঘ) শুধু YAML

20. একজন নতুন DevOps প্রকৌশলী হিসেবে প্রথম production pipeline বদলানোর আগে সবচেয়ে নিরাপদ অভ্যাস কোনটি?
ক) সব secret একসাথে বদলানো খ) কাউকে না জানানো গ) ছোট পরিবর্তন, PR ও review, staging-এ পরীক্ষা, rollback পরিকল্পনা — তারপর ধীরে ধীরে production-এ নেওয়া ঘ) সরাসরি production-এ বদলে ফেলা

নিজেকে যাচাই করুন: ১২০টির মধ্যে ৯০টির বেশি সঠিক হলে আপনি অধ্যায়গুলোর ধারণা ভালোভাবে ধরেছেন। ৭০-এর কম হলে ভুল হওয়া ব্লকের অধ্যায়গুলো আবার পড়ুন ও সংশ্লিষ্ট ল্যাব করুন। উত্তর আছে অংশ F-এ।

অংশ B — সংক্ষিপ্ত প্রশ্ন

৩০টি সংক্ষিপ্ত প্রশ্ন

প্রতিটি প্রশ্নের উত্তর ৩–৫ বাক্যে লিখুন। উত্তর লেখার আগে অধ্যায়ের সংজ্ঞাটি নিজের ভাষায় বলার চেষ্টা করুন। মডেল উত্তর আছে অংশ F-এ।

1. Continuous Integration, Continuous Delivery ও Continuous Deployment-এর মধ্যে পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন।
2. Release Engineering বলতে কী বোঝায়? একটি উদাহরণ দিন।
3. Source code, build output, release artifact ও deployed application — চারটির পার্থক্য লিখুন।
4. Pipeline-এ fail-fast নীতি কেন গুরুত্বপূর্ণ?
5. Idempotency ও repeatability বলতে পাইপলাইনে কী বোঝায়?
6. Trunk-based development-এর সুবিধা ও ঝুঁকি লিখুন।
7. GitHub Actions-এ workflow, job ও step-এর সম্পর্ক ব্যাখ্যা করুন।
8. Hosted runner বনাম self-hosted runner — কখন কোনটি বেছে নেবেন?
9. Reusable workflow ও composite action-এর পার্থক্য ও ব্যবহারের ক্ষেত্র লিখুন।
10. Matrix strategy কী এবং কখন এটি ব্যবহার করা উচিত?
11. GITHUB_TOKEN-এর permissions সীমিত রাখা কেন জরুরি? একটি উদাহরণ দিন।
12. OIDC কীভাবে দীর্ঘমেয়াদি secret-এর ঝুঁকি কমায়?
13. GitLab-এ stage, job, needs ও rules — প্রতিটির কাজ এক লাইনে লিখুন।
14. GitLab Runner-এ Docker executor বনাম Shell executor-এর সুবিধা-অসুবিধা লিখুন।
15. Jenkins-এ controller ও agent আলাদা রাখার কারণ ব্যাখ্যা করুন।
16. Declarative Pipeline ও Scripted Pipeline-এর মধ্যে কখন কোনটি বেছে নেবেন?
17. Unit, integration ও end-to-end test — প্রতিটির উদ্দেশ্য ও সীমাবদ্ধতা লিখুন।
18. Quality gate কী? একটি বাস্তব উদাহরণ দিন।
19. Code coverage বেশি হলেই কি কোড নিরাপদ? যুক্তি দিন।
20. Multi-stage Docker build কেন ছোট ও নিরাপদ image দেয়?
21. Image tag `latest` বনাম image digest — production-এ কোনটি ব্যবহার করবেন ও কেন?
22. Semantic Versioning-এর তিনটি অংশের অর্থ ও কখন প্রতিটি বাড়ানো হয়, লিখুন।
23. Blue-green, canary ও rolling deployment — প্রতিটির একটি সুবিধা ও একটি ঝুঁকি লিখুন।
24. Feature flag কীভাবে deploy ও release-কে আলাদা করে?
25. Database migration-এ expand-and-contract পদ্ধতিটি ব্যাখ্যা করুন।
26. Environment promotion-এ একই artifact ব্যবহার করা কেন গুরুত্বপূর্ণ?
27. Rollback পরিকল্পনা deploy-এর আগেই কেন তৈরি রাখতে হয়?
28. Pipeline-এ flaky test কীভাবে চিহ্নিত করবেন ও কী করবেন?
29. Pipeline governance বলতে কী বোঝায়? দুটি উদাহরণ দিন।

30. Blameless postmortem কী এবং এটি দলের সংস্কৃতিতে কীভাবে সাহায্য করে?

অংশ C — পরিস্থিতিভিত্তিক সমস্যা-সমাধান

২০টি পরিস্থিতিভিত্তিক ট্রাবলশুটিং প্রশ্ন

প্রতিটি পরিস্থিতি পড়ুন, তারপর লিখুন: (১) সম্ভাব্য কারণ কী, (২) কী কী যাচাই করবেন ও কোন ক্রমে, (৩) সমাধান কী, (৪) আবার যাতে না হয়, কী করবেন। মডেল উত্তর আছে অংশ F-এ।

1. **পরিস্থিতি:** প্রতিটি pull request-এ CI ব্যর্থ হয়, কিন্তু ডেভেলপারের নিজের কম্পিউটারে একই test পাস করে। **কী করবেন?**
2. **পরিস্থিতি:** GitHub Actions workflow হঠাৎ “Error: Unable to resolve action `some-action@v2`” দিয়ে ব্যর্থ হচ্ছে। গত সপ্তাহে ঠিক ছিল। **কী করবেন?**
3. **পরিস্থিতি:** Pipeline-এ `npm ci` ধাপে “lock file does not satisfy package.json” ত্রুটি আসছে। **কী করবেন?**
4. **পরিস্থিতি:** Docker image build ধাপে “no space left on device” ত্রুটি আসছে। **কী করবেন?**
5. **পরিস্থিতি:** Production deploy সফল দেখাচ্ছে, কিন্তু ব্যবহারকারীরা পুরনো সংস্করণই দেখছেন। **কী করবেন?**
6. **পরিস্থিতি:** Staging deploy-এর পর API ৫০০ ত্রুটি দিচ্ছে, কিন্তু CI-এর সব test পাস করেছিল। **কী করবেন?**
7. **পরিস্থিতি:** একই pipeline কখনো ৪ মিনিটে, কখনো ২৫ মিনিটে শেষ হয়। কোনো কোড পরিবর্তন নেই। **কী করবেন?**
8. **পরিস্থিতি:** GitLab pipeline-এ একটি job “stuck / pending” অবস্থায় আটকে আছে, কোনো runner নিচ্ছে না। **কী করবেন?**
9. **পরিস্থিতি:** Jenkins build “Out of memory” (Java heap) দিয়ে ব্যর্থ হচ্ছে। **কী করবেন?**
10. **পরিস্থিতি:** OIDC দিয়ে AWS-এ deploy করতে গিয়ে “AccessDenied — not authorized to perform sts:AssumeRoleWithWebIdentity” ত্রুটি আসছে। **কী করবেন?**
11. **পরিস্থিতি:** একটি secret ঘুরিয়ে (rotate) দেওয়ার পর সব deploy ব্যর্থ হচ্ছে। **কী করবেন?**
12. **পরিস্থিতি:** Pull request থেকে আসা কোড pipeline-এ secret ব্যবহার করতে পারছে না, অথচ main branch-এ পারে। **কী করবেন?**
13. **পরিস্থিতি:** Docker image registry-তে push করতে গিয়ে “denied: requested access to the resource is denied” ত্রুটি আসছে। **কী করবেন?**
14. **পরিস্থিতি:** Canary deploy-এর পর নতুন সংস্করণের error rate ধীরে ধীরে বাড়ছে, কিন্তু পুরনো সংস্করণ স্থিতিশীল। **কী করবেন?**
15. **পরিস্থিতি:** Database migration-সহ deploy-এর পর পুরনো কোডের কিছু অনুরোধ ব্যর্থ হচ্ছে। **কী করবেন?**
16. **পরিস্থিতি:** Pipeline-এ image scan-এ একটি critical CVE ধরা পড়ল, release আজই দিতে হবে বলে চাপ আছে। **কী করবেন?**
17. **পরিস্থিতি:** দুটি deploy একসাথে চলায় production-এ অর্ধেক pod নতুন, অর্ধেক পুরনো সংস্করণে চলছে। **কী করবেন?**
18. **পরিস্থিতি:** একজন ডেভেলপার ভুল করে production branch-এ সরাসরি push করে ফেলেছেন এবং pipeline স্বয়ংক্রিয়ভাবে deploy শুরু করেছে। **কী করবেন?**
19. **পরিস্থিতি:** Pipeline-এর test ধাপে MongoDB সংযোগ “connection refused” দিচ্ছে। **কী করবেন?**
20. **পরিস্থিতি:** সপ্তাহান্তে pipeline-এর খরচ হঠাৎ তিন গুণ বেড়ে গেছে, কোনো নতুন ফিচার deploy হয়নি। **কী করবেন?**

অংশ D — ইনসিডেন্ট কেস স্টাডি

১০টি CI/CD ইনসিডেন্ট কেস স্টাডি

প্রতিটি কেস একটি বাস্তব ধরনের ঘটনার উপর ভিত্তি করে লেখা (নাম, কোম্পানি ও সংখ্যা কাল্পনিক)। প্রথমে নিজে ভাবুন — কারণ কী হতে পারে, আপনি কী করতেন — তারপর অংশ F-এর সমাধানটি পড়ুন।

কেস ১ — “সবুজ পাইপলাইন, ভাঙা প্রোডাকশন”

ঘটনা: TechNova-র MERN অ্যাপের CI সবুজ ছিল, কিন্তু production deploy-এর ১০ মিনিট পর ব্যবহারকারীরা লগইন করতে পারছিলেন না। **লক্ষণ:** লগইন API ৫০০ ত্রুটি দিচ্ছিল; অন্য পাতাগুলো ঠিক ছিল। **তদন্ত:** deploy log-এ দেখা গেল নতুন environment variable `JWT_SECRET` production-এ সেট করা হয়নি; CI-তে test-এর জন্য একটি ডামি মান ব্যবহৃত হয়েছিল। **মূল কারণ:** environment variable-এর পার্থক্য CI-তে ধরা পড়েনি; production secret অনুপস্থিত ছিল। **সমাধান:** আগের সংস্করণে rollback, সঠিক secret সেট করে আবার deploy, health check-এ লগইন প্রবাহ যোগ করা। **প্রতিরোধ:** deploy-এর আগে প্রয়োজনীয় variable-এর চেকলিস্ট, staging-এ production-এর মতো secret (আলাদা মান) দিয়ে পরীক্ষা, smoke test-এ লগইন অন্তর্ভুক্ত করা।

কেস ২ — “ফাঁস হওয়া টোকেন”

ঘটনা: একজন ডেভেলপার ডিবাগ করতে গিয়ে workflow log-এ একটি API token print করেছিলেন; token-টি একটি public repository-র log-এ দৃশ্যমান ছিল। **লক্ষণ:** অস্বাভাবিক API ব্যবহার ধরা পড়ে। **তদন্ত:** log-এ token-টি খুঁজে পাওয়া যায়; token-টি দীর্ঘমেয়াদি ছিল। **মূল কারণ:** secret log-এ প্রকাশ করা ও দীর্ঘমেয়াদি token ব্যবহার। **সমাধান:** token সঙ্গে সঙ্গে বাতিল (revoke) ও নতুন token দেওয়া, log থেকে তথ্য সরানোর চেষ্টা, ক্ষতির পরিধি যাচাই। **প্রতিরোধ:** secret কখনো print না করা, secret masking চালু রাখা, স্বল্পমেয়াদি/OIDC credential ব্যবহার, secret scanning চালু করা।

কেস ৩ — “latest ট্যাগের ধোঁকা”

ঘটনা: Staging-এ পরীক্ষা করা image-টিই production-এ deploy করা হয়েছিল বলে ধরা হয়েছিল, কিন্তু production-এ অন্য কোড চলছিল। **লক্ষণ:** staging-এ ঠিক হওয়া বাগ production-এ আবার দেখা গেল। **তদন্ত:** দুটি environment-ই `latest` ট্যাগ ব্যবহার করেছিল; staging পরীক্ষার পর নতুন build `latest` বদলে দিয়েছিল। **মূল কারণ:** mutable tag ব্যবহার — কোন image চলছে তা নিশ্চিত ছিল না। **সমাধান:** digest দিয়ে deploy করা, tag-এ সংস্করণ নম্বর যোগ করা। **প্রতিরোধ:** production-এ সবসময় immutable tag বা digest ব্যবহার করা, environment promotion-এ একই artifact পুনঃব্যবহার করা।

কেস ৪ — “অপরিচিত PR-এর কোড production secret ছুঁয়ে ফেলল”

ঘটনা: একটি public repository-তে `pull_request_target` workflow অপরিচিত PR-এর কোড checkout করে secret-সহ পরিবেশে চালিয়েছিল। **লক্ষণ:** নিরাপত্তা পর্যালোচনায় ধরা পড়ে যে PR কোড secret-এর কাছাকাছি চলেছে। **তদন্ত:** workflow-টি PR কোড checkout করছিল ও secret ব্যবহার করছিল। **মূল কারণ:** `pull_request_target`-এর ঝুঁকি না বোঝা। **সমাধান:** workflow বদলে `pull_request` ইভেন্ট ব্যবহার, secret-যুক্ত ধাপ আলাদা করা, PR কোডে checkout না করা। **প্রতিরোধ:** PR workflow-এ least privilege, অনুমোদন ছাড়া secret ব্যবহার না করা, নিরাপত্তা পর্যালোচনা।

কেস ৫ — "মাইগ্রেশন যেটি পুরনো কোড ভেঙে দিল"

ঘটনা: একটি কলাম মুছে ফেলার migration ও নতুন কোড একসাথে deploy করা হলো; rolling deploy-এর সময় পুরনো pod-গুলো নতুন ডেটাবেস স্কিমায় ব্যর্থ হলো। **লক্ষণ:** কিছু অনুরোধ ৫০০ ত্রুটি দিচ্ছিল, কিছু ঠিক ছিল। **তদন্ত:** deploy-এর সময় পুরনো ও নতুন কোড একসাথে চলছিল, কিন্তু স্কিমা শুধু নতুন কোডের সঙ্গে সামঞ্জস্যপূর্ণ ছিল। **মূল কারণ:** backward-incompatible migration এক ধাপে করা। **সমাধান:** rollback, তারপর expand-and-contract পদ্ধতিতে দুই ধাপে migration করা। **প্রতিরোধ:** migration-এর আগে সামঞ্জস্য যাচাই, backward-compatible পরিবর্তন, deploy-এর আগে migration পরীক্ষা।

কেস ৬ — "ক্যাশ যেটি পুরনো কোড ধরে রাখল"

ঘটনা: একটি জরুরি নিরাপত্তা সংশোধন deploy করা হলো, কিন্তু production-এ পুরনো দুর্বল কোডই চলতে থাকল। **লক্ষণ:** সংস্করণ নম্বর বদলায়নি, scan-এ পুরনো CVE আবার ধরা পড়ল। **তদন্ত:** Docker layer cache ও dependency cache পুরনো layer পুনঃব্যবহার করেছিল; lockfile বদলালেও cache key বদলায়নি। **মূল কারণ:** ভুল cache key ও cache invalidation-এর অভাব। **সমাধান:** cache মুছে নতুন করে build করা, সঠিক cache key (lockfile hash) ব্যবহার। **প্রতিরোধ:** lockfile-ভিত্তিক cache key, নিরাপত্তা সংশোধনের পর cache-busting নীতি, image digest যাচাই।

কেস ৭ — "Jenkins controller যেটি থেমে গেল"

ঘটনা: সকালে সব Jenkins build আটকে গেল; controller-এর ডিস্ক পূর্ণ হয়ে গিয়েছিল। **লক্ষণ:** build queue বাড়ছিল, UI ধীর, log-এ disk full ত্রুটি। **তদন্ত:** পুরনো build artifact ও log জমে ডিস্ক ভরে গিয়েছিল; কোনো retention নীতি ছিল না। **মূল কারণ:** artifact/log retention ও ডিস্ক পর্যবেক্ষণের অভাব। **সমাধান:** পুরনো build মুছে ডিস্ক খালি করা, retention নীতি চালু করা, agent-এ build সরানো। **প্রতিরোধ:** build retention, ডিস্ক alert, controller-এ build না চালানো, নিয়মিত ব্যাকআপ।

কেস ৮ — "Flaky test যেটি release আটকে দিল"

ঘটনা: একটি গুরুত্বপূর্ণ release বারবার একই test-এ ব্যর্থ হচ্ছিল, কিন্তু কোডে কোনো পরিবর্তন ছিল না। **লক্ষণ:** test কখনো পাস, কখনো ফেল; ব্যর্থতা সবসময় timeout-এ। **তদন্ত:** test-টি একটি বাইরের API-এর উপর নির্ভর করছিল, যার প্রতিক্রিয়া সময় কখনো ধীর হতো। **মূল কারণ:** বাইরের নির্ভরতার উপর ভরসা করা flaky test। **সমাধান:** বাইরের API mock করা, timeout বাড়ানো নয় — test-কে নির্ভরযোগ্য করা, flaky test আলাদা করে চিহ্নিত করা। **প্রতিরোধ:** test isolation, mock/stub ব্যবহার, flaky test-এর তালিকা ও নিয়মিত পর্যালোচনা।

কেস ৯ — "দুটি deploy একসাথে"

ঘটনা: দুটি ডেভেলপার একই সময়ে production deploy করলেন; একটি deploy-এর অর্ধেক pod নতুন, অর্ধেক পুরনো সংস্করণে রইল। **লক্ষণ:** ব্যবহারকারীরা অনিয়মিত আচরণ দেখলেন; কিছু অনুরোধ নতুন, কিছু পুরনো সংস্করণে যাচ্ছিল। **তদন্ত:** concurrency নিয়ন্ত্রণ ছিল না; দুটি run একসাথে চলছিল। **মূল কারণ:** একই environment-এ একাধিক deploy একসাথে চলা। **সমাধান:** একটি deploy বাতিল করে অন্যটি সম্পূর্ণ করা, তারপর আবার deploy করা। **প্রতিরোধ:** concurrency control, deploy lock, একটি সময়ে একটি production deploy নীতি।

কেস ১০ — “স্ক্যান উপেক্ষা করার মূল্য”

ঘটনা: Image scan-এ একটি critical CVE ধরা পড়েছিল, কিন্তু সময়ের চাপে তা উপেক্ষা করে release দেওয়া হলো; পরে ওই দুর্বলতা কাজে লাগিয়ে আক্রমণ হলো। **লক্ষণ:** অস্বাভাবিক নেটওয়ার্ক কার্যকলাপ, সন্দেহজনক প্রক্রিয়া। **তদন্ত:** scan রিপোর্টে CVE-টি আগেই ছিল; quality gate-এ scan ব্যর্থতাকে বাধ্যতামূলক করা হয়নি। **মূল কারণ:** নিরাপত্তা gate-কে ঐচ্ছিক রাখা ও চাপের কাছে নতি স্বীকার। **সমাধান:** আক্রান্ত image সরানো, patch করা image deploy করা, ক্ষতির পরিধি যাচাই। **প্রতিরোধ:** critical CVE-তে pipeline ব্যর্থ করা (blocking gate), ব্যতিক্রমের জন্য লিখিত অনুমোদন প্রক্রিয়া, নিয়মিত dependency আপডেট।

অংশ E — চূড়ান্ত পরীক্ষা

ব্যবহারিক চূড়ান্ত পরীক্ষা

মোট নম্বর: ১০০ | **সময়:** ৩ ঘণ্টা | **পাস নম্বর:** ৬০ | **উপকরণ:** ল্যাপটপ, ইন্টারনেট, GitHub/GitLab অ্যাকাউন্ট, Docker। পরীক্ষার সময় অন্যের কাজ দেখা বা কপি করা যাবে না। তত্ত্ব অংশে বই বন্ধ রাখুন; হাতে-কলমে অংশে ডকুমেন্টেশন দেখা যাবে।

> **পরীক্ষার নিয়ম:** Section A শেষ করে তবেই Section B শুরু করুন। Section C-তে কোনো production সার্ভার বা আসল secret ব্যবহার করবেন না — শুধু নিজের test repository ও স্থানীয় Docker ব্যবহার করুন। কাজ শেষে repository-র লিঙ্ক ও স্ক্রিনশট জমা দিন।

Section A — তত্ত্ব (৩০ নম্বর, ৪৫ মিনিট)

নিচের ৬টি প্রশ্নের প্রতিটির উত্তর ৫–৮ বাক্যে লিখুন। প্রতিটি ৫ নম্বর।

1. Continuous Integration, Continuous Delivery ও Continuous Deployment-এর পার্থক্য ব্যাখ্যা করুন এবং একটি ছোট দলের জন্য কোনটি কখন উপযুক্ত, তা যুক্তিসহ লিখুন। (৫)
2. Pipeline-এ quality gate কী? অন্তত চারটি gate-এর নাম লিখুন এবং একটি gate ব্যর্থ হলে কী হওয়া উচিত, তা ব্যাখ্যা করুন। (৫)
3. GitHub Actions-এ `pull_request` ও `pull_request_target`-এর পার্থক্য ও নিরাপত্তা ঝুঁকি ব্যাখ্যা করুন। (৫)
4. Blue-green, canary ও rolling deployment-এর তুলনা করুন — প্রতিটির সুবিধা, ঝুঁকি ও উপযুক্ত পরিস্থিতি লিখুন। (৫)
5. Image tag `latest` বনাম digest — production deploy-এ কোনটি নিরাপদ ও কেন? Environment promotion-এর সঙ্গে এর সম্পর্ক লিখুন। (৫)
6. OIDC কী এবং এটি দীর্ঘমেয়াদি secret-এর চেয়ে কেন নিরাপদ? একটি উদাহরণ দিয়ে ব্যাখ্যা করুন। (৫)

Section B — পাইপলাইন ডিজাইন কাজ (৩৫ নম্বর, ১ ঘণ্টা ১৫ মিনিট)

প্রেক্ষাপট: TechNova-র একটি MERN অ্যাপ আছে — React frontend, Express backend, MongoDB। Repository-তে `client/` ও `server/` ফোল্ডার আছে। দলটি চায়: প্রতিটি pull request-এ lint, test ও build যাচাই; main branch-এ merge হলে Docker image তৈরি, scan ও registry-তে প্রকাশ; staging-এ স্বয়ংক্রিয় deploy; production-এ অনুমোদনের পর deploy; deploy-এর পর health check ও প্রয়োজনে rollback।

কাজ:

1. পাইপলাইনের আর্কিটেকচার ডায়াগ্রাম আঁকুন (হাতে বা ডিজিটাল) — trigger, stage, job, environment ও অনুমোদন দেখান। (৮)
2. GitHub Actions-এর সম্পূর্ণ workflow YAML লিখুন — PR validation, lint, test, build, Docker build/scan/push, staging deploy, production approval ও health check সহ। YAML-এ কোনো আসল secret লিখবেন না; প্লেসহোল্ডার ব্যবহার করুন। (১৫)
3. প্রতিটি stage-এর উদ্দেশ্য ও quality gate এক অনুচ্ছেদে ব্যাখ্যা করুন — কেন এই stage আছে, ব্যর্থ হলে কী হবে। (৭)

4. Rollback পরিকল্পনা লিখুন — কখন rollback করবেন, কীভাবে করবেন, কে সিদ্ধান্ত নেবে, এবং rollback-এর পর কী যাচাই করবেন। (৫)

মূল্যায়ন: YAML সিনট্যাক্সের চেয়ে ডিজাইনের যুক্তি, নিরাপত্তা (least privilege, secret ব্যবস্থাপনা), environment separation ও rollback-এর স্পষ্টতা বেশি গুরুত্ব পাবে।

Section C — হাতে-কলমে ট্রাবলশুটিং কাজ (৩৫ নম্বর, ১ ঘণ্টা)

প্রেক্ষাপট: আপনাকে একটি ইচ্ছাকৃতভাবে ভাঙা (broken) উদাহরণ repository দেওয়া হবে, যেখানে নিচের তিনটি সমস্যার মধ্যে অন্তত দুটি আছে — (ক) CI-তে test ব্যর্থ হচ্ছে, (খ) Docker build ব্যর্থ হচ্ছে, (গ) deploy-এর পর health check ব্যর্থ হচ্ছে।

কাজ:

1. প্রথম সমস্যাটি নির্ণয় করুন — log পড়ে মূল কারণ লিখুন ও ঠিক করুন। (১২)
2. দ্বিতীয় সমস্যাটি নির্ণয় করুন — log পড়ে মূল কারণ লিখুন ও ঠিক করুন। (১২)
3. আপনার নির্ণয় ও সমাধানের একটি সংক্ষিপ্ত প্রতিবেদন লিখুন — কী ছিল, কীভাবে খুঁজলেন, কী বদলালেন, আবার যাতে না হয় কী করবেন। (৬)
4. ঠিক করা পাইপলাইনের সবুজ (সফল) run-এর প্রমাণ জমা দিন — run-এর লিঙ্ক বা স্ক্রিনশট। (৫)

মূল্যায়ন: শুধু “ঠিক হয়েছে” যথেষ্ট নয় — log পড়ে কারণ খুঁজে বের করার প্রক্রিয়া, সমাধানের যুক্তি ও প্রতিরোধের পরিকল্পনা মূল্যায়ন করা হবে।

> **পরীক্ষকের নির্দেশ:** Section C-এর ভাঙা repository পরীক্ষার আগে তৈরি রাখুন — প্রতিটি সমস্যার মূল কারণ ও সমাধান আলাদা নোটে লিখে রাখুন। পরীক্ষার্থীকে সরাসরি উত্তর দেবেন না; শুধু log পড়তে ও ধাপে ধাপে ভাবতে সাহায্য করুন।

অংশ F — উত্তর নির্দেশিকা

উত্তর নির্দেশিকা ও ক্যাপস্টোন রুব্রিক

এই অংশটি পড়ার আগে নিজে চেষ্টা করুন। প্রতিটি MCQ উত্তরের পাশে এক লাইনের ব্যাখ্যা দেওয়া আছে — অক্ষরটি নয়, ব্যাখ্যাটিই শেখার জিনিস।

MCQ উত্তর ১–৬০

1. **গ** — Release Engineering হলো build, test, package ও release-এর নির্ভরযোগ্য প্রক্রিয়া নকশা ও পরিচালনা।
2. **গ** — ঘন ঘন integration ও স্বয়ংক্রিয় test-এ সংঘাত ও ত্রুটি তাড়াতাড়ি ধরা পড়ে।
3. **খ** — Delivery-তে মানুষের অনুমোদনের পর production-এ যাওয়া যায়; Deployment-এ তা স্বয়ংক্রিয়।
4. **খ** — ছোট, ঘন ঘন merge-এ conflict কমে ও সমস্যা তাড়াতাড়ি ধরা পড়ে।
5. **খ** — Build output কাঁচা ফল, artifact সংস্করণসহ প্রকাশযোগ্য প্যাকেজ, deployed app চলমান সংস্করণ।
6. **গ** — Fail-fast মানে ব্যর্থতা ধরা পড়লেই থেমে যাওয়া, যাতে ত্রুটিপূর্ণ কোড এগিয়ে না যায়।
7. **খ** — Idempotency মানে একই ইনপুটে বারবার একই ফল, দ্বিতীয়বার চালালে ক্ষতি হয় না।
8. **গ** — Trunk-based development-এ ছোট পরিবর্তন দ্রুত main-এ মেশে, drift কমে।
9. **ঘ** — PR CI merge-এর আগেই test, lint ও build যাচাই করে।
10. **ক** — push সরাসরি কমিটে চলে, pull_request merge-এর আগে প্রস্তাবিত পরিবর্তনে চলে।
11. **গ** — build → test → deploy ক্রমটি একটি quality gate; test না পেরিয়ে deploy বিপজ্জনক।
12. **গ** — Path-based trigger শুধু বদলানো অংশের pipeline চালায়।
13. **খ** — Versioning ছাড়া কোন build কোথায় চলছে ও কোথায় ফিরতে হবে, তা জানা যায় না।
14. **খ** — দীর্ঘ feature branch-এ merge conflict ও drift বাড়ে।
15. **গ** — Observability মানে log, status ও মেট্রিক দেখে ব্যর্থতার কারণ বোঝা।
16. **ঘ** — Tag নির্দিষ্ট কমিটকে স্থায়ীভাবে চিহ্নিত করে, release পুনরাবৃত্তিযোগ্য হয়।
17. **খ** — ভাঙা build দীর্ঘদিন থাকলে সবাই অনিশ্চিত কোডের উপর কাজ করে।
18. **ক** — Stage dependency মানে আগের stage-এর ফল পরের stage-এর ইনপুট।
19. **খ** — Automated pipeline পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য ও ভুল কম হয়।
20. **খ** — প্রথম CI-তে install, lint ও unit test — সহজ, দ্রুত ও নির্ভরযোগ্য গেট।
21. **গ** — Artifact repository সংস্করণসহ প্যাকেজ সংরক্ষণ ও পুনঃব্যবহারের জন্য।
22. **গ** — Lockfile সব পরিবেশে একই dependency সংস্করণ নিশ্চিত করে।
23. **গ** — Workflow-এ job থাকে, job-এ step থাকে; job ডিফল্টভাবে সমান্তরালে চলে।
24. **ঘ** — Hosted runner GitHub পরিচালনা করে; self-hosted-এ নিয়ন্ত্রণ বেশি, দায়িত্বও বেশি।
25. **ঘ** — অপরিচিত PR কোড self-hosted runner-এ চললে নেটওয়ার্ক ও secret ঝুঁকিতে পড়ে।
26. **গ** — checkout action repository-র কোড runner-এ নামিয়ে আনে।
27. **ক** — Reusable workflow job স্তরে পুরো workflow ডাকে; composite action step স্তরে একাধিক step একত্র করে।
28. **গ** — needs দিয়ে job-এর নির্ভরতা ও ক্রম নিয়ন্ত্রণ করা হয়।
29. **ক** — Matrix একই job একাধিক Node সংস্করণ বা OS-এ চালিয়ে সামঞ্জস্য যাচাই করে।

30. **ক** — `if: failure()` আগের step ব্যর্থ হলেই চলে।
31. **ঘ** — Least privilege মানে শুধু প্রয়োজনীয় অনুমতি; আপস হলে ক্ষতি সীমিত থাকে।
32. **ঘ** — Third-party action-এর উৎস, pin করা সংস্করণ ও অনুমতি যাচাই করা উচিত।
33. **খ** — SHA pin করলে tag বদলে গেলেও একই কোড চলে, supply-chain ঝুঁকি কমে।
34. **গ** — Caching বড় dependency সংরক্ষণ করে install সময় কমায়।
35. **খ** — Lockfile hash-ভিত্তিক cache key dependency বদলালে পুরনো cache ব্যবহার আটকায়।
36. **খ** — Artifact upload job-এর ফল অন্য job বা পরে ডাউনলোডের জন্য সংরক্ষণ করে।
37. **গ** — Concurrency একই branch-এ একাধিক run একসাথে চলা আটকায়।
38. **ক** — pull_request_target base repo-র secret নিয়ে চলে, তাই PR কোডে checkout ঝুঁকিপূর্ণ।
39. **খ** — Environment protection rule production deploy-এর আগে reviewer অনুমোদন বাধ্যতামূলক করে।
40. **ক** — ব্যর্থ হলে প্রথমে কোন job/step ব্যর্থ হয়েছে ও তার log দেখতে হয়।
41. **গ** — Log-এ secret থাকলে তা সংরক্ষিত ও দৃশ্যমান হয়ে ফাঁস হতে পারে।
42. **ঘ** — OIDC-তে দীর্ঘমেয়াদি secret নয়, প্রতিটি run-এর জন্য স্বল্পমেয়াদি token তৈরি হয়।
43. **খ** — Branch-ভিত্তিক trigger সীমিত করলে অপ্রয়োজনীয় run কমে, খরচ ও সময় বাঁচে।
44. **খ** — এটি context থেকে পাওয়া বর্তমান pull request-এর নম্বর।
45. **ক** — `.gitlab-ci.yml`-এ pipeline-এর stage, job, script ও নিয়ম ঘোষণা করা হয়।
46. **ক** — Stage ধাপের ক্রম, job সেই stage-এর কাজ; একই stage-এর job সমান্তরালে চলে।
47. **গ** — Docker executor পরিষ্কার পরিবেশ ও ভালো isolation দেয়; Shell executor-এ পরিবেশ দূষণের ঝুঁকি থাকে।
48. **খ** — rules দিয়ে কোন branch/tag-এ job চলবে তা নিয়ন্ত্রণ করা হয়।
49. **গ** — needs ক্রম ভেঙে আগের job শেষ হলেই পরের job চালায়; dependencies artifact নির্বাচন করে।
50. **গ** — Protected variable শুধু protected branch/tag-এর pipeline-এ ব্যবহারযোগ্য।
51. **খ** — Environment stop action অস্থায়ী environment মুছে সম্পদ মুক্ত করে।
52. **ঘ** — include দিয়ে একাধিক project-এ একই pipeline অংশ পুনঃব্যবহার করা যায়।
53. **খ** — Cache পরের run দ্রুত করার অস্থায়ী সংরক্ষণ; artifact job-এর সংরক্ষিত ফল।
54. **খ** — Manual job production deploy-এর মতো সংবেদনশীল ধাপে মানুষের অনুমোদন নেয়।
55. **গ** — বিশেষ পরিবেশ বা নিরাপত্তা প্রয়োজন হলে নির্দিষ্ট project-এর জন্য specific runner দরকার।
56. **ঘ** — Optimization-এর বড় লাভ অপ্রয়োজনীয় job বাদ দেওয়া, cache ও সমান্তরাল চালানো থেকে আসে।
57. **খ** — Flaky test কোড না বদলালেও কখনো পাস, কখনো ফেল করে, সাধারণত timing বা বাইরের নির্ভরতার কারণে।
58. **ঘ** — workflow:rules দিয়ে পুরো pipeline কখন তৈরি হবে তা নিয়ন্ত্রণ করা হয়।
59. **ক** — setup-node-এর cache অপশন npm dependency cache করে `npm ci` দ্রুত করে।
60. **ক** — ধীর pipeline-এ প্রথমে lockfile-ভিত্তিক cache ও `npm ci` চালু করতে হয়।

MCQ উত্তর ৬১–১২০

1. **ঘ** — Controller pipeline পরিচালনা করে, agent আসল build/test কাজ চালায়।
2. **ক** — Controller-এ build চালালে সম্পদ শেষ হয়, নিরাপত্তা ঝুঁকি বাড়ে ও controller অস্থিতিশীল হয়।
3. **ঘ** — Declarative-তে নির্দিষ্ট গঠন থাকে ও পড়া সহজ; Scripted-তে Groovy-র পূর্ণ নমনীয়তা থাকে।

4. **গ** — post failure ব্লক pipeline ব্যর্থ হলে নোটিফিকেশন বা cleanup চালায়।
5. **ঘ** — Credentials Jenkins Credentials Store-এ রেখে ID দিয়ে ব্যবহার করতে হয়, কোডে নয়।
6. **গ** — Shared Library একাধিক pipeline-এ একই ধাপ পুনঃব্যবহার করতে দেয়।
7. **খ** — Plugin-এর রক্ষণাবেক্ষণ, নিরাপত্তা সতর্কতা ও সংস্করণ সামঞ্জস্য যাচাই করতে হয়।
8. **খ** — Docker agent মানে ওই stage নির্দিষ্ট Docker image-এর ভেতরে চলবে।
9. **ক** — Unit test একটি অংশ আলাদাভাবে যাচাই করে; integration test একাধিক অংশ একসাথে যাচাই করে।
10. **ঘ** — E2E test ধীর ও ভঙ্গুর হতে পারে, তাই অল্প সংখ্যায় গুরুত্বপূর্ণ প্রবাহে ব্যবহার করা উচিত।
11. **গ** — Coverage শুধু কোন লাইন চলেছে তা বলে, যুক্তি সঠিক কি না তা বলে না।
12. **ক** — Linting স্টাইল ও সম্ভাব্য ভুল তাড়াতাড়ি ধরে, কোড একরূপ রাখে।
13. **খ** — Dependency scan লাইব্রেরিতে জানা নিরাপত্তা দুর্বলতা খোঁজে।
14. **খ** — Secret scanning ভুল করে কমিট করা token merge-এর আগে ধরে।
15. **ক** — Quality gate মানে নির্দিষ্ট মান না পেরোলে পরের ধাপে যাওয়া যাবে না।
16. **গ** — SAST কোড না চালিয়ে বিশ্লেষণ করে; DAST চলমান অ্যাপ্লিকেশন পরীক্ষা করে।
17. **ঘ** — Type checking runtime-এর আগেই টাইপ-সংক্রান্ত ভুল ধরে।
18. **ক** — সমান্তরাল test একই ডেটাবেস বা ফাইল ভাগ করলে একে অপরের ফল নষ্ট করতে পারে।
19. **ক** — Pipeline parameter একই pipeline বিভিন্ন ইনপুট দিয়ে চালাতে দেয়।
20. **খ** — Test pyramid-এ সবচেয়ে বেশি unit test, তারপর integration, সবচেয়ে কম E2E।
21. **খ** — Multi-stage build build-এর সরঞ্জাম বাদ দিয়ে ছোট, নিরাপদ production image দেয়।
22. **খ** — `latest` পরিবর্তনশীল, তাই কোন সংস্করণ চলছে তা নিশ্চিত করা যায় না।
23. **গ** — Digest অপরিবর্তনীয়, তাই ঠিক একই image সবসময় টানা হয়।
24. **ক** — Image scan image-এর ভেতরের প্যাকেজ ও লাইব্রেরিতে জানা CVE খোঁজে।
25. **ক** — Registry authentication শুধু অনুমোদিত pipeline-কেই image প্রকাশ করতে দেয়।
26. **খ** — Immutable artifact একবার তৈরি হলে একই সংস্করণ সব environment-এ ব্যবহৃত হয়।
27. **ঘ** — SemVer-এ major বড় পরিবর্তন, minor নতুন ফিচার, patch ছোট সংশোধন।
28. **খ** — Environment promotion-এ একই artifact dev → staging → production-এ যাচাই করে এগিয়ে নেওয়া হয়।
29. **খ** — Blue-green-এ দুটি পরিবেশ প্রস্তুত রেখে ট্রাফিক একবারে বদলানো হয়, ফেরা সহজ।
30. **গ** — Canary-তে নতুন সংস্করণ প্রথমে অল্প ব্যবহারকারীর উপর পরীক্ষা করা হয়।
31. **গ** — Rolling deploy-এ কিছুক্ষণ পুরনো ও নতুন সংস্করণ একসাথে চলে, অসামঞ্জস্যের ঝুঁকি থাকে।
32. **গ** — Feature flag দিয়ে deploy আর release আলাদা করা যায়।
33. **খ** — Expand-and-contract-এ প্রথমে নতুন ক্ষেত্র যোগ করা হয়, পরে পুরনোটি সরানো হয়।
34. **ক** — Health check নতুন সংস্করণ প্রস্তুত কি না যাচাই করে তবেই ট্রাফিক দেয়।
35. **ক** — Rollback পরিকল্পনা আগেই থাকলে সমস্যায় দ্রুত ও চাপ ছাড়া ফেরা যায়।
36. **ক** — Release candidate হলো চূড়ান্ত release-এর আগের পরীক্ষামূলক সংস্করণ।
37. **গ** — Approval gate staging যাচাই ও ঝুঁকি দেখে অনুমোদন নিশ্চিত করে।
38. **খ** — আগের layer না বদলালে Docker layer caching সেগুলো পুনঃব্যবহার করে।
39. **খ** — `.dockerignore` অপ্রয়োজনীয় ফাইল বাদ দিয়ে build দ্রুত ও নিরাপদ করে।

40. **গ** — Smoke test deploy-এর পর মূল প্রবাহগুলো দ্রুত যাচাই করে।
41. **ক** — Frontend ও backend আলাদা job-এ সমান্তরালে test/build হয়, ব্যর্থতা আলাদাভাবে ধরা যায়।
42. **ঘ** — Integration test-এর জন্য service container বা অস্থায়ী MongoDB দিয়ে বিচ্ছিন্ন পরিবেশ দরকার।
43. **ক** — CI-তে `npm ci` lockfile অনুযায়ী পরিষ্কারভাবে ইনস্টল করে, পুনরাবৃত্তিযোগ্য build দেয়।
44. **ঘ** — ধীর pipeline-এ প্রথমে কোন step সবচেয়ে বেশি সময় নিচ্ছে, তা চিহ্নিত করতে হয়।
45. **ক** — একই নির্দিষ্ট পরিবেশে build/test হলে “works on my machine” সমস্যা কমে।
46. **গ** — Governance মানে কে pipeline বদলাতে পারে, কোন অনুমোদন লাগে, secret কীভাবে ব্যবহৃত হয় — এসব নিয়ম।
47. **ক** — Code review, branch protection, secret scanning ও least privilege secret print করা আটকায়।
48. **খ** — Audit trail কে, কখন, কী deploy করল ও কোন অনুমোদন ছিল, তা সংরক্ষণ করে।
49. **ক** — Deploy-এর মতো side-effect ধাপে অন্ধ retry-তে একই কাজ দুবার হতে পারে।
50. **খ** — Pipeline সংজ্ঞা Git-এ সংস্করণসহ থাকলে হারিয়ে গেলে আবার তৈরি করা যায়।
51. **ঘ** — টুল বেছে নেওয়ার সময় বিদ্যমান প্ল্যাটফর্ম, রক্ষণাবেক্ষণের ক্ষমতা, নিরাপত্তা ও খরচ দেখতে হয়।
52. **ক** — খরচ নিয়ন্ত্রণে অপ্রয়োজনীয় trigger কমানো, cache করা ও ব্যর্থ run তাড়াতাড়ি থামানো কার্যকর।
53. **গ** — Alert পেলে প্রথমে ব্যর্থতার পরিধি বুঝে runbook অনুযায়ী কাজ করতে হয়।
54. **ক** — Blameless postmortem-এ দোষারোপ না করে প্রক্রিয়া ও ব্যবস্থার দুর্বলতা খুঁজে সংশোধন করা হয়।
55. **ক** — Shift left মানে নিরাপত্তা ও quality পরীক্ষা যত তাড়াতাড়ি সম্ভব নিয়ে আসা।
56. **গ** — Concurrency নিয়ন্ত্রণ না থাকলে একই commit দুবার deploy হতে পারে।
57. **ক** — Release note-এ কী বদলেছে, কী ঝুঁকি আছে ও কীভাবে rollback করতে হয়, তা থাকা উচিত।
58. **ক** — Error rate বাড়লে প্রস্তুত rollback পথে আগের স্থিতিশীল সংস্করণে ফেরাই সেরা প্রথম পদক্ষেপ।
59. **খ** — Pipeline documentation-এ আর্কিটেকচার, trigger, secret, অনুমোদন, rollback ও যোগাযোগের পথ থাকা উচিত।
60. **গ** — প্রথম production পরিবর্তনে ছোট পরিবর্তন, review, staging পরীক্ষা ও rollback পরিকল্পনা সবচেয়ে নিরাপদ।

> **উত্তর-বণ্টন নিয়ে নোট:** এই সংস্করণে বেশিরভাগ সঠিক উত্তর “খ” বিকল্পে এসেছে, কারণ প্রশ্ন লেখার সময় সঠিক ব্যাখ্যাটি দ্বিতীয় বিকল্পে রাখা হয়েছিল। ভবিষ্যৎ সংস্করণে বিকল্পগুলোর ক্রম এলোমেলো করা হবে। পরীক্ষায় অক্ষর অনুমান করে নয়, ব্যাখ্যা বুঝে উত্তর দিন।

অংশ F (চলমান) — মডেল উত্তর

সংক্ষিপ্ত প্রশ্নের মডেল উত্তর (১–৩০)

1. **CI:** পরিবর্তন ঘন ঘন একত্র করে স্বয়ংক্রিয় build/test। **Delivery:** যেকোনো সময় deploy-এর জন্য প্রস্তুত, তবে production-এ যেতে মানুষের অনুমোদন লাগে। **Deployment:** সফল পরিবর্তন স্বয়ংক্রিয়ভাবে production-এ যায়, মানুষের অনুমোদন লাগে না। পার্থক্যটি অনুমোদনের জায়গায়।
2. Release Engineering হলো কোডকে নির্ভরযোগ্যভাবে build, test, package, version ও release করার প্রক্রিয়া নকশা ও পরিচালনা। উদাহরণ: প্রতিটি merge-এ সংস্করণ নম্বরসহ Docker image তৈরি, scan করে registry-তে প্রকাশ ও staging-এ deploy করা।
3. Source code হলো মানুষের লেখা কোড; build output হলো কম্পাইল/বান্ডল করা ফল; release artifact হলো সংস্করণসহ সংরক্ষিত প্রকাশযোগ্য প্যাকেজ; deployed application হলো কোনো environment-এ চলমান সেই artifact-এর সংস্করণ।
4. Fail-fast মানে ব্যর্থতা ধরা পড়লেই pipeline থেমে যাওয়া। এতে ত্রুটিপূর্ণ কোড পরের ধাপে যায় না, সময় ও সম্পদ বাঁচে এবং সমস্যার উৎস খুঁজে পাওয়া সহজ হয়।
5. Idempotency মানে একই ইনপুটে বারবার চালালে একই ফল, দ্বিতীয়বার চালালে ক্ষতি হয় না। Repeatability মানে একই ইনপুটে যেকোনো সময় একই build পাওয়া যায় — lockfile ও নির্দিষ্ট সংস্করণ এর ভিত্তি।
6. Trunk-based development-এ ছোট পরিবর্তন দ্রুত main-এ মেশে, তাই conflict ও drift কমে এবং CI দ্রুত প্রতিক্রিয়া দেয়। ঝুঁকি: অসম্পূর্ণ ফিচার main-এ ঢুকতে পারে, তাই feature flag ও ভালো test দরকার।
7. Workflow হলো একটি YAML ফাইলে সংজ্ঞায়িত স্বয়ংক্রিয় প্রক্রিয়া; তাতে একাধিক job থাকে; প্রতিটি job-এ একাধিক step থাকে। Job ডিফল্টভাবে সমান্তরালে চলে, step একই job-এ ক্রমানুসারে চলে।
8. Hosted runner পরিচালনার ঝামেলা নেই, প্রতিবার পরিষ্কার পরিবেশ দেয় — বেশিরভাগ দলের জন্য উপযুক্ত। Self-hosted runner বিশেষ হার্ডওয়্যার, নেটওয়ার্ক বা নিরাপত্তা প্রয়োজন হলে, তবে রক্ষণাবেক্ষণ ও isolation নিজের দায়িত্ব।
9. Reusable workflow পুরো workflow অন্য workflow থেকে job স্তরে ডাকে; composite action একাধিক step একত্র করে একটি step হিসেবে ব্যবহার করে। বড়, পুনরাবৃত্ত প্রক্রিয়ায় reusable workflow, ছোট ধাপের গুচ্ছে composite action।
10. Matrix strategy একই job একাধিক Node সংস্করণ, OS বা কনফিগারেশনে সমান্তরালে চালায়। সামঞ্জস্য যাচাইয়ে উপযুক্ত; খরচ ও সময় বাড়ে বলে শুধু প্রয়োজনীয় সংমিশ্রণে ব্যবহার করা উচিত।
11. GITHUB_TOKEN-এর অনুমতি বেশি থাকলে workflow-এর কোনো দুর্বলতা বা আপস হওয়া action repository বদলে ফেলতে পারে। তাই শুধু প্রয়োজনীয় অনুমতি দিন — যেমন শুধু contents: read, প্রয়োজনে আলাদাভাবে লিখুন।
12. OIDC-তে দীর্ঘমেয়াদি secret সংরক্ষণ করতে হয় না; প্রতিটি run-এর জন্য identity যাচাই করে স্বল্পমেয়াদি token তৈরি হয়, মেয়াদ শেষ হলে তা অকেজো হয়ে যায়। ফাঁস হলেও ক্ষতির সময়সীমা কম থাকে।
13. Stage হলো ধাপের ক্রম; job হলো stage-এর ভেতরের কাজ; needs দিয়ে stage-এর ক্রম ভেঙে নির্ভরতা বলা যায়; rules দিয়ে কোন শর্তে job চলবে তা বলা যায়।
14. Docker executor প্রতিটি job-এ পরিষ্কার image-ভিত্তিক পরিবেশ দেয়, isolation ভালো, তবে Docker জানতে হয়। Shell executor সহজ, কিন্তু আগের job-এর অবশিষ্টাংশ পরিবেশ দূষিত করতে পারে।
15. Controller pipeline পরিচালনা, সময়সূচি ও UI দেয়; agent build/test চালায়। আলাদা রাখলে build controller-এর সম্পদ খায় না, নিরাপত্তা ভালো হয় এবং build স্কেল করা যায়।

16. Declarative Pipeline-এ নির্দিষ্ট গঠন থাকে, পড়া ও রক্ষণাবেক্ষণ সহজ — বেশিরভাগ কাজের জন্য উপযুক্ত। Scripted Pipeline-এ Groovy-র পূর্ণ নমনীয়তা থাকে, তবে জটিল ও ভুলপ্রবণ — শুধু বিশেষ প্রয়োজনে।
17. Unit test একটি অংশ আলাদাভাবে যাচাই করে — দ্রুত ও অনেক; integration test অংশগুলোর একসাথে কাজ যাচাই করে; E2E test পুরো প্রবাহ যাচাই করে — ধীর ও ভঙ্গুর। তিনটিরই সীমা আছে, একটিই অন্যটির বিকল্প নয়।
18. Quality gate হলো নির্দিষ্ট মান না পেরোলে পরের ধাপে যাওয়া যাবে না — এমন শর্ত। উদাহরণ: সব test পাস, lint পরিষ্কার, critical CVE নেই — এই তিনটি না হলে deploy হবে না।
19. না। Coverage শুধু কোন লাইন চলেছে তা বলে; যুক্তি সঠিক কি না, ভুল ইনপুটে কী হয়, তা বলে না। ৯০% coverage-এর test-ও ভুল assertion থাকলে ত্রুটি ধরতে পারে না।
20. Multi-stage build-এ প্রথম stage-এ build করা হয়, চূড়ান্ত stage-এ শুধু প্রয়োজনীয় ফল কপি করা হয়। Build-এর সরঞ্জাম চূড়ান্ত image-এ থাকে না, তাই image ছোট হয় ও আক্রমণের পৃষ্ঠ (attack surface) কমে।
21. Production-এ digest ব্যবহার করবেন, কারণ digest অপরিবর্তনীয় — ঠিক একই image চলবে। `latest` পরিবর্তনশীল, তাই কোন সংস্করণ চলছে তা নিশ্চিত করা যায় না।
22. SemVer-এ major বড়, backward-incompatible পরিবর্তনে; minor নতুন ফিচারে (সামঞ্জস্য রেখে); patch ছোট বাগ সংশোধনে বাড়ানো হয়। যেমন 1.4.2 থেকে বড় পরিবর্তনে 2.0.0।
23. Blue-green: ফেরা সহজ, খরচ বেশি। Canary: ঝুঁকি ধীরে ধীরে যাচাই করা যায়, পর্যবেক্ষণ জটিল। Rolling: সম্পদ কম লাগে, কিছুক্ষণ দুটি সংস্করণ একসাথে চলে।
24. Feature flag দিয়ে কোড deploy করা আর ফিচার চালু করা আলাদা করা যায়। কোড production-এ গেলেও flag বন্ধ থাকলে ফিচারটি ব্যবহারকারীরা দেখে না; পরে flag চালু করলেই ফিচারটি আসে, নতুন deploy লাগে না।
25. Expand-and-contract-এ প্রথমে নতুন কলাম/ক্ষেত্র যোগ করা হয় (expand), কোড দুটি স্কিমার সঙ্গে কাজ করে, তারপর পুরনো ক্ষেত্র সরানো হয় (contract)। এতে পুরনো ও নতুন কোড একসাথে চললেও ভাঙে না।
26. একই artifact সব environment-এ ব্যবহার করলে staging-এ যাচাই করা জিনিসটিই production-এ যায়। নতুন করে build করলে পরিবেশের পার্থক্যে ভিন্ন ফল আসতে পারে, তখন staging পরীক্ষা অর্থহীন হয়ে যায়।
27. সমস্যা হলে দ্রুত ও চাপ ছাড়া আগের সংস্করণে ফিরতে হয়; ঘটনার সময় পরিকল্পনা বানানো ধীর ও ভুলপ্রবণ। আগে থেকে rollback ধাপ, দায়িত্ব ও যাচাই তালিকা থাকলে পুনরুদ্ধার দ্রুত হয়।
28. Flaky test চিহ্নিত করতে একই কোডে test বারবার চালান ও ব্যর্থতার ধরন দেখুন — timing, বাইরের নির্ভরতা বা ভাগ করা সম্পদ সন্দেহ করুন। সমাধান: mock করা, isolation দেওয়া, প্রয়োজনে test আলাদা করে চিহ্নিত করে ঠিক করা।
29. Pipeline governance হলো pipeline-এর নিয়ম ও নিয়ন্ত্রণ — কে YAML বদলাতে পারে, কোন অনুমোদন লাগে, secret কীভাবে ব্যবহৃত হয়। উদাহরণ: branch protection ও production deploy-এ reviewer অনুমোদন।
30. Blameless postmortem-এ ঘটনার পর কাউকে দোষ না দিয়ে প্রক্রিয়া ও ব্যবস্থার দুর্বলতা খোঁজা হয়। এতে মানুষ ভুল স্বীকার করতে ভয় পায় না, সত্যিকারের কারণ বেরিয়ে আসে ও ব্যবস্থা উন্নত হয়।

পরিস্থিতিভিত্তিক প্রশ্নের মডেল উত্তর (১–২০)

1. **কারণ:** পরিবেশের পার্থক্য — Node সংস্করণ, dependency সংস্করণ বা environment variable। **যাচাই:** CI log-এর ত্রুটি, Node সংস্করণ, lockfile আছে কি না, `npm ci` বনাম `npm install`। **সমাধান:** lockfile কমিট করা, CI-তে নির্দিষ্ট Node সংস্করণ, Docker-ভিত্তিক পরিবেশ। **প্রতিরোধ:** স্থানীয়ভাবে CI-এর মতো পরিবেশে test চালানো।
2. **কারণ:** action-এর tag বদলে গেছে, action মুছে গেছে বা নেটওয়ার্ক সমস্যা। **যাচাই:** action-এর repository ও সংস্করণ, workflow-এ pin করা আছে কি না, GitHub status। **সমাধান:** commit SHA বা স্থিতিশীল সংস্করণে pin করা, প্রয়োজনে বিকল্প action। **প্রতিরোধ:** সব action pin করা ও নিয়মিত পর্যালোচনা।

3. **কারণ:** `package.json` বদলেছে কিন্তু lockfile আপডেট হয়নি। **যাচাই:** কোন dependency বদলেছে, lockfile-এর অবস্থা। **সমাধান:** স্থানীয়ভাবে `npm install` করে lockfile আপডেট ও কমিট করা। **প্রতিরোধ:** lockfile কমিট বাধ্যতামূলক করা, PR-এ lockfile যাচাই।
4. **কারণ:** পুরনো image, layer বা cache ডিস্ক ভরে ফেলেছে। **যাচাই:** ডিস্ক ব্যবহার, Docker-এর অব্যবহৃত সম্পদ। **সমাধান:** `docker system prune` (সতর্কতার সঙ্গে), পুরনো image মুছে ফেলা, ডিস্ক বাড়ানো। **প্রতিরোধ:** নিয়মিত cleanup, retention নীতি।
5. **কারণ:** cache, পুরনো image বা ভুল environment-এ deploy। **যাচাই:** চলমান image-এর digest/সংস্করণ, deploy log, CDN বা browser cache। **সমাধান:** সঠিক digest দিয়ে আবার deploy, cache পরিষ্কার করা। **প্রতিরোধ:** deploy-এর পর সংস্করণ যাচাই (smoke test)।
6. **কারণ:** environment পার্থক্য — staging-এর variable, ডেটাবেস বা বাইরের সেবা। **যাচাই:** অ্যাপের log, environment variable-এর তালিকা, ডেটাবেস সংযোগ। **সমাধান:** অনুপস্থিত variable সেট করা, প্রয়োজনে rollback। **প্রতিরোধ:** deploy-পূর্ব variable চেকলিস্ট, staging-এ production-এর মতো কনফিগারেশন।
7. **কারণ:** cache miss, runner-এর লোড, নেটওয়ার্ক বা বাইরের সেবার ধীরগতি। **যাচাই:** প্রতিটি step-এর সময়, cache hit/miss, runner-এর ধরন। **সমাধান:** cache ঠিক করা, ধীর step আলাদা করা। **প্রতিরোধ:** pipeline সময়ের মেট্রিক ও alert।
8. **কারণ:** runner অফলাইন, tag মিলছে না বা runner ব্যস্ত। **যাচাই:** runner-এর অবস্থা, job-এর tag, runner-এর log। **সমাধান:** runner চালু করা, tag ঠিক করা, shared runner ব্যবহার। **প্রতিরোধ:** runner health পর্যবেক্ষণ।
9. **কারণ:** Jenkins heap ছোট বা memory leak। **যাচাই:** heap সেটিং, build-এর মেমরি ব্যবহার, plugin। **সমাধান:** heap বাড়ানো, controller-এ build না চালিয়ে agent-এ চালানো। **প্রতিরোধ:** মেমরি পর্যবেক্ষণ ও সীমা নির্ধারণ।
10. **কারণ:** IAM role-এর trust policy-তে ভুল repository, branch বা audience। **যাচাই:** trust policy, OIDC provider, workflow-এর অনুমতি। **সমাধান:** trust policy-তে সঠিক repository/branch ও audience লেখা। **প্রতিরোধ:** OIDC কনফিগারেশন নথিভুক্ত করা ও পরীক্ষা করা।
11. **কারণ:** নতুন secret সব জায়গায় আপডেট হয়নি বা ভুল মান দেওয়া হয়েছে। **যাচাই:** কোন environment-এ secret বদলেছে, পুরনো secret-এর মেয়াদ। **সমাধান:** সঠিক secret সব environment-এ সেট করা, প্রয়োজনে পুরনো secret ফিরিয়ে আনা (যদি নিরাপদ হয়)। **প্রতিরোধ:** rotation চেকলিস্ট, একবারে একটি environment-এ বদলানো।
12. **কারণ:** Fork করা PR-এ secret দেওয়া হয় না (নিরাপত্তা নীতি)। **যাচাই:** PR-টি fork থেকে কি না, workflow-এর ইভেন্ট। **সমাধান:** secret-নির্ভর ধাপ শুধু trusted branch-এ চালানো, PR-এ mock ব্যবহার। **প্রতিরোধ:** PR workflow-এ secret ব্যবহারের নীতি নথিভুক্ত করা।
13. **কারণ:** registry login হয়নি, ভুল registry বা অনুমতি নেই। **যাচাই:** login ধাপ, registry নাম, token-এর অনুমতি। **সমাধান:** সঠিক credential দিয়ে login করা, registry নাম ঠিক করা। **প্রতিরোধ:** OIDC বা স্বল্পমেয়াদি credential ব্যবহার।
14. **কারণ:** নতুন কোডে ত্রুটি বা সামঞ্জস্যের সমস্যা। **যাচাই:** error log, মেট্রিক, নতুন কোডের পরিবর্তন। **সমাধান:** canary বন্ধ করে পুরনো সংস্করণে ট্রাফিক ফেরানো (rollback)। **প্রতিরোধ:** canary-এর স্বয়ংক্রিয় rollback শর্ত নির্ধারণ করা।
15. **কারণ:** backward-incompatible migration। **যাচাই:** migration-এর পরিবর্তন, পুরনো কোডের ত্রুটি। **সমাধান:** rollback, expand-and-contract পদ্ধতিতে আবার করা। **প্রতিরোধ:** migration-এর সামঞ্জস্য যাচাই।
16. **কারণ:** জানা critical দুর্বলতাসহ image। **যাচাই:** CVE-এর গুরুত্ব ও প্রভাব। **সমাধান:** release থামানো, patch করা বা দুর্বলতা দূর করা; জরুরি হলে লিখিত ব্যতিক্রম ও সময়সীমা নিয়ে এগোনো। **প্রতিরোধ:** critical CVE-তে blocking gate।

17. **কারণ:** concurrency নিয়ন্ত্রণ নেই। **যাচাই:** চলমান deploy-এর সংখ্যা, concurrency সেটিং। **সমাধান:** একটি deploy বাতিল করে অন্যটি সম্পূর্ণ করা। **প্রতিরোধ:** concurrency control ও deploy lock।
18. **কারণ:** branch protection নেই বা trigger ভুল। **যাচাই:** branch protection, workflow trigger। **সমাধান:** deploy থামানো, কোড পর্যালোচনা করে প্রয়োজনে rollback। **প্রতিরোধ:** main branch-এ সরাসরি push বন্ধ করা, PR বাধ্যতামূলক করা।
19. **কারণ:** MongoDB service চালু হয়নি বা সংযোগ তথ্য ভুল। **যাচাই:** service container-এর অবস্থা, সংযোগ string, পোর্ট। **সমাধান:** service ঠিক করা, health check-এর জন্য অপেক্ষা করা। **প্রতিরোধ:** service-এর readiness যাচাই।
20. **কারণ:** অপ্রয়োজনীয় trigger, ব্যর্থ run-এর পুনরাবৃত্তি বা বড় runner। **যাচাই:** run-এর সংখ্যা, trigger, runner-এর সময়। **সমাধান:** অপ্রয়োজনীয় trigger বন্ধ করা, cache চালু করা, ছোট runner ব্যবহার। **প্রতিরোধ:** খরচের alert ও নিয়মিত পর্যালোচনা।

কেস স্টাডি সমাধান — সারাংশ

অংশ D-এর প্রতিটি কেসের মূল কারণ, সমাধান ও প্রতিরোধ সেখানেই দেওয়া আছে। এখানে মনে রাখার মূল শিক্ষা: (১) CI সবুজ মানেই production নিরাপদ নয় — environment পার্থক্য যাচাই করুন। (২) Secret কখনো log-এ প্রকাশ করবেন না; স্বল্পমেয়াদি credential ব্যবহার করুন। (৩) Production-এ mutable tag নয়, digest ব্যবহার করুন। (৪) অপরিচিত PR কোড secret-সহ পরিবেশে চালাবেন না। (৫) Migration সবসময় backward-compatible রাখুন। (৬) Cache key lockfile-ভিত্তিক রাখুন। (৭) Jenkins-এ retention ও ডিস্ক পর্যবেক্ষণ রাখুন। (৮) Flaky test-কে mock ও isolation দিয়ে ঠিক করুন। (৯) এক সময়ে একটি production deploy চালান। (১০) Critical CVE-তে gate বাধ্যতামূলক করুন।

চূড়ান্ত পরীক্ষার মার্কিং গাইড

Section A — তত্ত্ব (৩০ নম্বর)

প্রতিটি প্রশ্ন ৫ নম্বর: সঠিক সংজ্ঞা ২, পার্থক্য বা তুলনা ২, উদাহরণ বা যুক্তি ১। শুধু সংজ্ঞা লিখলে সর্বোচ্চ ২। ভুল ধারণা থাকলে ০–১।

Section B — পাইপলাইন ডিজাইন (৩৫ নম্বর)

মানদণ্ড	নম্বর	ভালো উত্তরের লক্ষণ
আর্কিটেকচার ডায়াগ্রাম	৮	Trigger, stage, job, environment ও অনুমোদন স্পষ্ট; ক্রম যৌক্তিক
Workflow YAML	১৫	PR validation, lint, test, build, scan, staging, approval, health check আছে; secret প্লেসহোল্ডার; অনুমতি সীমিত
Stage ব্যাখ্যা ও gate	৭	প্রতিটি stage-এর উদ্দেশ্য ও ব্যর্থতার পরিণতি ব্যাখ্যা করা হয়েছে
Rollback পরিকল্পনা	৫	কখন, কীভাবে, কে সিদ্ধান্ত নেবে ও কী যাচাই করবে — চারটিই আছে

Section C — হাতে-কলমে ট্রাবলশুটিং (৩৫ নম্বর)

প্রতিটি সমস্যা ১২ নম্বর: মূল কারণ নির্ণয় ৬, সঠিক সমাধান ৪, প্রমাণ (সবুজ run) ২। প্রতিবেদন ৬ নম্বর: কী ছিল ২, কীভাবে খুঁজলেন ২, প্রতিরোধ ২। সবুজ run-এর প্রমাণ ৫ নম্বর। শুধু "ঠিক হয়েছে" লিখলে সমাধানের নম্বর মিলবে না — log-ভিত্তিক কারণ দেখাতে হবে।

> **পরীক্ষকের নোট:** YAML সিনট্যাক্সের ছোট ভুলে পুরো নম্বর কাটবেন না, যদি ডিজাইনের যুক্তি সঠিক থাকে। নিরাপত্তা ভুল (যেমন secret কোডে লেখা, production-এ অনুমোদন ছাড়া deploy) থাকলে সংশ্লিষ্ট অংশে নম্বর কমবে।

ক্যাপস্টোন গ্রেডিং রুব্রিক (মোট ১০০ নম্বর)

TechNova MERN ডেলিভারি প্ল্যাটফর্ম — পাঠক DevOps Engineer হিসেবে যা তৈরি করবেন, তা এই রুব্রিকে মূল্যায়ন করা হবে। প্রতিটি মানদণ্ডে ৪টি স্তর: চমৎকার (৯০–১০০%), ভালো (৭০–৮৯%), গ্রহণযোগ্য (৫০–৬৯%), অপর্যাপ্ত (০–৪৯%)।

মানদণ্ড	নম্বর	চমৎকার স্তরের প্রমাণ (পরিমাপযোগ্য)
PR-ভিত্তিক workflow	১০	প্রতিটি PR-এ lint, test, build চলে; branch protection চালু; merge-এর আগে সব gate পাস বাধ্যতামূলক
স্বয়ংক্রিয় build ও test	১০	Unit + integration test pipeline-এ চলে; test ব্যর্থ হলে merge আটকায়; coverage রিপোর্ট তৈরি হয়
Code quality gate	১০	ESLint, TypeScript check ও dependency scan pipeline-এ আছে; critical সমস্যায় pipeline ব্যর্থ হয়
নিরাপদ credential ব্যবস্থাপনা	১০	কোনো আসল secret কোডে নেই; OIDC বা স্বল্পমেয়াদি credential; অনুমতি least privilege; secret scanning চালু
Docker image ও registry	১০	Multi-stage build; সংস্করণ tag + digest; image scan; registry-তে সফল push-এর প্রমাণ
Staging deployment	১০	Staging-এ স্বয়ংক্রিয় deploy; health check পাস; smoke test-এর প্রমাণ
অনুমোদন-ভিত্তিক production deploy	১০	Production environment-এ reviewer অনুমোদন বাধ্যতামূলক; অনুমোদন ছাড়া deploy হয় না — প্রমাণসহ
Health check ও rollback	১০	Deploy-এর পর health check; ব্যর্থতায় rollback ধাপ নথিভুক্ত ও পরীক্ষিত (rollback-এর প্রমাণ বা অনুশীলনের রেকর্ড)
Release versioning ও নথি	১০	SemVer tag; release note (পরিবর্তন, ঝুঁকি, rollback ধাপ); pipeline নথি (আর্কিটেকচার, trigger, secret, অনুমোদন)
নথি ও উপস্থাপনা	১০	README-এ আর্কিটেকচার, চালানোর ধাপ ও ট্রাবলশুটিং আছে; সব প্রমাণ (run লিঙ্ক/স্ক্রিনশট) জমা দেওয়া হয়েছে

পাস নম্বর: ৬০। **Distinction:** ৮৫ বা তার বেশি। নিরাপত্তা মানদণ্ডে (credential ব্যবস্থাপনা) ৫-এর কম পেলে সামগ্রিকভাবে পাস দেওয়া যাবে না, এমনকি মোট ৬০ হলেও — কারণ নিরাপদ credential ছাড়া production pipeline গ্রহণযোগ্য নয়।